

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Харківський національний університет радіоелектроніки

Факультет _____ комп'ютерних наук (або центр післядипломної освіти, або навчально-науковий центр заочної форми навчання)
(повна назва)

Кафедра _____ програмної інженерії
(повна назва)

КОМПЛЕКСНИЙ КУРСОВИЙ ПРОЄКТ
Пояснювальна записка

рівень вищої освіти _____ перший (бакалаврський)

(тема)

Виконав:

здобувач (ка) 3 курсу, групи ПЗП-23-5

Андрій ЖИЛЄНКОВ

(Власне ім'я, ПРІЗВИЩЕ)

Спеціальність 121 – Інженерія програмного
забезпечення

(код і повна назва спеціальності)

Тип програми _____ освітньо-професійна

Освітня програма Програмна інженерія

(повна назва освітньої програми)

Керівник доцент кафедри ПІ Андрій РАБОТЯГОВ

(посада, Власне ім'я, ПРІЗВИЩЕ)

Члени комісії (Власне ім'я, ПРІЗВИЩЕ, підпис)

Володимир КОБЗЄВ

Віктор КАУК

Дмитро КОЛЕСНИКОВ

2026 р.

Харківський національний університет радіоелектроніки

Факультет _____ комп'ютерних наук (або центр післядипломної освіти, або навчально-науковий центр заочної форми навчання)
 Кафедра _____ програмної інженерії
 Рівень вищої освіти _____ перший (бакалаврський)
 Спеціальність _____ 121 – Інженерія програмного забезпечення
 Тип програми _____ Освітньо-професійна
 Освітня програма _____ Програмна Інженерія
 (шифр і назва)

Курс 3Група ПЗП-23-5Семестр 6

ЗАВДАННЯ
на курсовий проект(роботу) студента

здобувачеві _____ Жилєнкову Андрію Олександровичу
 (прізвище, ім'я, по батькові)

1. Тема роботи _____ Програмна система контролю якості виробництва лінз за зображенням їх поверхні

2. Термін здачі студентом закінченої роботи „1” червня 2026 р.

3. Вихідні _____ дані _____ до _____ проекту

4. Перелік питань, що потрібно опрацювати в роботі

КАЛЕНДАРНИЙ ПЛАН

№	Назва етапів роботи	Термін виконання етапів роботи	Примітка
1	Аналіз предметної галузі	06.03-12.03	виконано
2	Розробка постановки задачі	12.03-20.03	виконано
3	Проектування ПЗ	20.03-30.03	виконано
4	Програмна реалізація	30.03-25.04	виконано
5	Аналіз результатів	25.04-26.04	виконано
6	Підготовка пояснювальної записки.	26.04-27.05	виконано
7	Перевірка на наявність ознак академічного плагіату		виконано
8	Захист роботи		виконано

Дата видачі завдання “_6_” _березня_ 2026р.

Здобувач _____
(підпис)

Керівник роботи _____ доцент кафедри ПІ Андрій РАБОТЯГОВ
(підпис) (посада, Власне ім'я, ПРІЗВИЩЕ)

РЕФЕРАТ / ABSTRACT

Пояснювальна записка містить: 61 стор., 7 рис., 14 джерел.

АВТОМАТИЗОВАНИЙ КОНТРОЛЬ ЯКОСТІ ЛІНЗ, ВИЯВЛЕННЯ ДЕФЕКТІВ, КОМП'ЮТЕРНИЙ ЗІР, ОБРОБКА ЗОБРАЖЕНЬ, ПРОГРАМНИЙ ЗАСТОСУНОК, OPENCV, PYTHON, PYSIDE6

Об'єкт розробки – програмний застосунок для автоматизованого контролю якості оптичних лінз.

Мета розробки – створення програмного застосунку для автоматичного виявлення дефектів на поверхні лінз за допомогою методів комп'ютерного зору та цифрової обробки зображень.

Метод рішення – використання мови програмування Python, бібліотек OpenCV та PySide6, методів фільтрації, аналізу контурів, пошуку меж лінзи, покращення контрасту та детекції пошкоджень поверхні.

У результаті розробки створено програмний застосунок автоматизованого контролю якості лінз, який забезпечує аналіз зображень, виявлення дефектів поверхні, автоматичне сортування придатних і бракованих лінз, фоновий моніторинг папки із зображеннями, а також генерацію PDF-звітів з результатами аналізу.

AUTOMATED LENS QUALITY CONTROL, COMPUTER VISION, DEFECT DETECTION, IMAGE PROCESSING, OPENCV, PROGRAM SOFTWARE, PYTHON, PYSIDE6

The object of development is a software application for automated quality control of optical lenses.

The purpose of the work is to create a software application for automatic detection of defects on lens surfaces using computer vision and digital image processing methods.

Solution method – usage of the Python programming language, OpenCV and PySide6 libraries, filtering methods, contour analysis, lens boundary detection, contrast enhancement, and surface defect detection techniques.

As a result of the development, a software application for automated lens quality control was created, which provides image analysis, surface defect detection, automatic sorting of acceptable and defective lenses, background folder monitoring with images, and generation of PDF reports with analysis results.

ЗМІСТ

Перелік скорочень.....	7
Вступ.....	8
1 Аналіз предметної галузі.....	9
1.1 Аналіз предметної галузі.....	9
1.2 Виявлення та вирішення проблем.....	12
1.2.1 Цільова аудиторія.....	15
1.3 Аналіз аналогів програмного забезпечення.....	16
2 Постановка задачі.....	18
3 Архітектура та проєктування програмного забезпечення.....	23
3.1 UML проєктування ПЗ.....	23
3.2 Проєктування архітектури програмного забезпечення.....	26
3.3 Створення UI/UX.....	29
4 Опис прийнятих програмних рішень.....	33
4.1 Обробка зображення.....	33
4.2 Автоматичний моніторинг.....	39
5 Аналіз отриманих результатів.....	41
Висновки.....	43
Перелік джерел посилання.....	44
Додаток А.....	46
Додаток Б.....	49
Додаток В.....	55

ПЕРЕЛІК СКОРОЧЕНЬ

API – Application Programming Interface
BMP – Bitmap Image File
CLAHE – Contrast Limited Adaptive Histogram Equalization
CPU – Central Processing Unit
GUI – Graphical User Interface
JPEG – Joint Photographic Experts Group
PDF – Portable Document Format
PNG – Portable Network Graphics
PySide6 – Python Bindings for the Qt Framework
Qt – Cross-platform Application Framework
RGB – Red Green Blue
ROI – Region of Interest
UI – User Interface
OpenCV – Open Source Computer Vision Library
IDE – Integrated Development Environment
JSON – JavaScript Object Notation
HoughLinesP – Probabilistic Hough Line Transform
Canny – Canny Edge Detection Algorithm
NumPy – Numerical Python
PDF – Portable Document Format
QThread – Qt Thread
QtCore – Qt Core Module
QtWidgets – Qt Widgets Module
UTF – Unicode Transformation Format

ВСТУП

У сучасному виробництві оптичних елементів особливу увагу приділяють контролю якості продукції, оскільки навіть незначні дефекти поверхні лінз можуть суттєво впливати на їхні характеристики та подальшу експлуатацію. Традиційні методи перевірки лінз часто потребують значних часових витрат, участі оператора та не виключають можливості людської помилки. Саме тому актуальним є впровадження автоматизованих систем контролю якості, що базуються на методах комп'ютерного зору та цифрової обробки зображень. Використання сучасних програмних засобів дозволяє автоматизувати процес аналізу поверхні лінз, підвищити точність виявлення дефектів та забезпечити стабільність результатів перевірки. Застосування алгоритмів обробки зображень дає можливість визначати подряпини, пошкодження, неоднорідності поверхні та інші дефекти без необхідності постійного втручання людини. Крім того, автоматизовані системи можуть працювати у безперервному режимі, що особливо важливо для виробничих процесів із великим потоком продукції.

Метою роботи є розробка програмного застосунку для автоматизованого контролю якості оптичних лінз із використанням методів комп'ютерного зору та цифрової обробки зображень. Програмний застосунок повинен забезпечувати автоматичне виявлення дефектів поверхні лінз, аналіз отриманих зображень, сортування придатних і бракованих виробів, а також формування звітів за результатами проведеної перевірки. Для реалізації програмного застосунку використовувалися мова програмування Python, бібліотеки OpenCV для обробки зображень та PySide6 для створення графічного інтерфейсу користувача. Для формування PDF-звітів використовувалася бібліотека ReportLab. Результати роботи можуть бути використані у виробничих системах контролю якості оптичної продукції, навчальних цілях, а також у програмних системах, пов'язаних із комп'ютерним зором та автоматизованим аналізом зображень.

1 АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ

1.1 Аналіз предметної галузі

У сучасних умовах розвитку промисловості питання контролю якості продукції набуває особливої актуальності. Постійне зростання вимог до точності виробництва, надійності продукції та швидкості виробничих процесів змушує підприємства впроваджувати нові технології автоматизації. Особливо це стосується сфер, пов'язаних із виготовленням оптичних виробів, де навіть незначні дефекти поверхні можуть призводити до суттєвого погіршення характеристик готової продукції.

Лінзи є важливою складовою великої кількості сучасних пристроїв та систем. Вони використовуються в окулярній оптиці, фототехніці, медичних приладах, мікроскопах, лазерних установках, камерах відеоспостереження, смартфонах, телескопах та інших високоточних оптичних системах. Від якості поверхні лінзи безпосередньо залежить точність передачі світла, рівень спотворень зображення, чіткість фокусування та загальна ефективність роботи обладнання.

У процесі виробництва та транспортування лінзи можуть зазнавати різноманітних механічних пошкоджень. Найбільш поширеними дефектами є подряпини, тріщини, сколи, потертості, нерівномірності покриття, мікродефекти поверхні та локальні забруднення. Навіть незначна подряпина може стати причиною погіршення оптичних характеристик виробу. У високоточних системах це може призводити до серйозних помилок вимірювання або спотворення зображення.

Однією з основних особливостей контролю якості оптичних виробів є складність виявлення дефектів. Поверхня лінзи має прозору структуру та часто створює відблиски, які ускладнюють візуальний аналіз. Крім того, дефекти можуть мати дуже малий розмір і бути практично непомітними для людського ока. Саме тому контроль якості у даній сфері потребує високої точності та стабільності результатів. Тривалий час основним методом перевірки лінз залишався ручний

контроль. Працівник виконує огляд поверхні за допомогою спеціального освітлення та оптичних пристроїв збільшення. Незважаючи на поширеність такого підходу, він має низку суттєвих недоліків. Найбільшою проблемою є людський фактор. Якість перевірки значною мірою залежить від досвіду оператора, його уважності, фізичного стану та рівня втоми. Під час тривалої роботи точність виявлення дефектів може суттєво знижуватися.

Ще однією особливістю ручного контролю є низька продуктивність. У сучасному виробництві кількість продукції може досягати тисяч одиниць на добу, тому ручна перевірка потребує значних часових витрат. Крім того, збільшення швидкості роботи оператора часто призводить до зростання кількості помилок та пропущених дефектів. Окрему складність становить контроль мікродефектів. Людське око не завжди здатне стабільно розпізнавати дрібні подряпини або незначні пошкодження поверхні. Особливо це проявляється при нестабільному освітленні або наявності відблисків. Через це навіть якісний ручний контроль не може гарантувати абсолютну точність результатів.

У зв'язку з цим у сучасному світі активно розвиваються системи автоматизованого контролю якості. Такі системи дозволяють мінімізувати вплив людського фактору та забезпечити стабільність аналізу. Основою подібних рішень є технології комп'ютерного зору та цифрової обробки зображень.

Комп'ютерний зір є одним із напрямків штучного інтелекту та автоматизації, який дозволяє аналізувати цифрові зображення за допомогою програмних алгоритмів. Системи комп'ютерного зору вже широко застосовуються у промисловості для сортування продукції, контролю геометричних параметрів, виявлення дефектів поверхні та автоматичного аналізу виробничих процесів. З розвитком обчислювальної техніки та алгоритмів обробки зображень з'явилася можливість створювати системи, здатні виконувати складний аналіз у реальному часі. Такі системи можуть працювати безперервно протягом тривалого часу, підтримувати високу швидкість перевірки та забезпечувати стабільну якість аналізу незалежно від зовнішніх факторів.

Особливо актуальним є використання автоматизованих систем у галузі контролю оптичних поверхонь. На відміну від людини, програмні алгоритми здатні аналізувати зображення з однаковою точністю протягом необмеженого часу. Крім того, автоматизовані рішення дозволяють виконувати перевірку значно швидше та зменшують витрати на виробничий контроль. Складність аналізу лінз полягає не лише у виявленні дефектів, а й у необхідності правильно визначити межі самого об'єкта. Лінза на зображенні може мати різний рівень освітлення, нахил або часткові затемнення. У деяких випадках контури поверхні є недостатньо чіткими, що значно ускладнює сегментацію області аналізу. Ще однією особливістю предметної області є різноманітність типів дефектів. Подряпини можуть бути короткими або довгими, прямими або вигнутими, мати різний рівень контрасту та розташовуватися під будь-яким кутом. Деякі дефекти помітні лише при певному освітленні. Через це створення універсальної системи контролю є складною задачею, яка потребує застосування різних методів цифрової обробки зображень.

У сучасних умовах виробництва також зростає потреба у безперервному моніторингу продукції. Багато підприємств переходять до концепції автоматизованих виробничих ліній, де контроль якості має виконуватися без участі людини. У таких умовах система повинна самостійно отримувати нові зображення, виконувати аналіз та формувати результат перевірки. Важливою складовою сучасних систем контролю є також візуалізація результатів. Оператор повинен мати можливість швидко переглянути результати аналізу, оцінити виявлені дефекти та отримати інформацію про стан продукції. Це особливо важливо у випадках, коли необхідно проводити додаткову перевірку або формувати технічну документацію. Окрім цього, сучасні виробничі підприємства потребують збереження історії аналізу та формування звітності. Наявність звітів дозволяє вести статистику браку, контролювати якість виробництва та документувати результати перевірок. У багатьох випадках така документація є необхідною умовою сертифікації продукції та внутрішнього контролю підприємства. Розвиток технологій Industry 4.0 [1] та цифровізації виробництва ще

більше підвищує актуальність автоматизованих систем контролю якості. Підприємства прагнуть створювати повністю автоматизовані виробничі процеси, у яких участь людини мінімізується. У таких умовах програмні системи комп'ютерного зору стають невід'ємною складовою сучасного виробництва.

1.2 Виявлення та вирішення проблем

Сучасне виробництво оптичної продукції характеризується високими вимогами до якості виробів та швидкості їх виготовлення. У сфері виробництва лінз навіть незначні дефекти поверхні можуть негативно впливати на експлуатаційні характеристики готової продукції. Саме тому контроль якості оптичних поверхонь є однією з ключових задач виробничого процесу. Незважаючи на активний розвиток автоматизації, на багатьох підприємствах перевірка якості лінз усе ще виконується вручну. Подібний підхід має значну кількість недоліків та створює серйозні проблеми як для виробництва, так і для кінцевого споживача продукції.

Однією з головних проблем є людський фактор. Результати перевірки значною мірою залежать від уважності, досвіду та фізичного стану оператора. Навіть висококваліфікований працівник не здатний підтримувати однакову концентрацію протягом тривалого часу. Через втому, монотонність роботи та перевантаження можуть виникати помилки під час аналізу поверхні лінз. У результаті частина дефектних виробів може потрапляти до готової продукції.

Особливо гостро ця проблема проявляється під час перевірки великих партій виробів. При масовому виробництві оператори змушені аналізувати сотні або навіть тисячі лінз за зміну. У таких умовах імовірність пропуску дефектів значно зростає. Крім того, швидкість ручного контролю часто не відповідає сучасним темпам виробництва.

Ще однією серйозною проблемою є складність виявлення мікродефектів. Подряпини на поверхні лінз можуть бути дуже тонкими та майже непомітними для людського ока. Часто дефекти мають слабкий контраст або перекриваються

відблисками від освітлення. Це ускладнює їх своєчасне виявлення навіть при використанні збільшувального обладнання.

Окрему проблему створює нестабільність умов освітлення. Під час перевірки поверхня лінзи може відбивати світло під різними кутами, утворюючи відблиски та тіні. Через це дефекти можуть виглядати по-різному залежно від положення виробу або джерела світла. У деяких випадках подряпина помітна лише під певним кутом огляду.

Складність контролю також полягає у різноманітності типів дефектів. Пошкодження поверхні можуть мати різну форму, розмір та глибину. Деякі дефекти є чітко вираженими, тоді як інші майже зливаються з поверхнею лінзи. Універсальні методи перевірки не завжди здатні стабільно визначати всі типи пошкоджень.

Важливою проблемою сучасного виробництва є також висока вартість помилок контролю якості. Потрапляння дефектної лінзи до готової продукції може призводити до:

- погіршення якості оптичних систем;
- виникнення спотворень зображення;
- зниження точності вимірювального обладнання;
- втрати довіри споживачів;
- фінансових збитків підприємства;
- необхідності відкликання продукції.

У галузях, пов'язаних із медициною, науковими дослідженнями або високоточною технікою, подібні помилки можуть мати особливо серйозні наслідки. Саме тому сучасні підприємства прагнуть максимально мінімізувати ризики пропуску дефектів.

Ще однією проблемою є відсутність безперервного контролю продукції. Ручна перевірка потребує постійної присутності оператора та не може

виконуватися цілодобово без втрати ефективності. У сучасних виробничих умовах виникає потреба у системах, здатних працювати у режимі 24/7 та автоматично реагувати на появу нової продукції.

Також актуальною проблемою є складність сортування результатів перевірки. На багатьох підприємствах процес розподілу продукції на придатну та браковану виконується вручну. Це уповільнює виробничий процес та створює додаткове навантаження на персонал.

Окремої уваги потребує проблема документування результатів аналізу. У багатьох випадках інформація про проведений контроль або не зберігається взагалі, або фіксується вручну. Такий підхід ускладнює:

- ведення статистики браку;
- аналіз ефективності виробництва;
- аудит якості продукції;
- контроль виробничих процесів;
- формування технічної документації.

У сучасних умовах підприємства потребують автоматизованих систем, які дозволять не лише виявляти дефекти, а й зберігати результати перевірок у зручному вигляді для подальшого аналізу.

Розвиток цифрових технологій та концепції Industry 4.0 створює додаткові вимоги до автоматизації виробництва. Сучасні підприємства прагнуть інтегрувати інтелектуальні системи контролю у виробничі лінії для забезпечення максимальної ефективності роботи. У таких умовах особливо актуальними стають технології комп'ютерного зору та автоматичної обробки зображень.

Ще однією проблемою є недостатня доступність спеціалізованих систем контролю якості для невеликих підприємств. Багато професійних рішень мають високу вартість та потребують складного обладнання. Через це частина

виробників не має можливості впроваджувати сучасні системи автоматизованого аналізу.

Також важливо враховувати, що сучасні виробничі процеси потребують максимальної швидкості прийняття рішень. У разі виявлення дефектної продукції система повинна оперативно реагувати та мінімізувати потрапляння браку до наступних етапів виробництва.

Усі зазначені проблеми свідчать про необхідність створення сучасних автоматизованих засобів контролю якості оптичних виробів. Подібні рішення повинні забезпечувати високу точність аналізу, стабільність роботи, можливість безперервного моніторингу та зручну взаємодію з користувачем.

1.2.1 Цільова аудиторія

Основною цільовою аудиторією майбутнього рішення є підприємства та організації, діяльність яких пов'язана з виробництвом, перевіркою або використанням оптичних виробів. До таких користувачів можуть належати:

- виробники окулярних лінз;
- підприємства з виготовлення фото- та відеооптики;
- виробники медичного обладнання;
- лабораторії контролю якості;
- сервісні центри оптичної техніки;
- науково-дослідні установи;
- підприємства високоточного виробництва.

Також потенційними користувачами можуть бути навчальні заклади та дослідницькі центри, які займаються вивченням технологій комп'ютерного зору та цифрової обробки зображень. Окрему категорію користувачів становлять невеликі підприємства, які потребують доступного інструменту автоматизованого контролю якості без необхідності придбання дорогих промислових комплексів.

1.3 Аналіз аналогів програмного забезпечення

Сучасні системи автоматизованого контролю якості широко застосовуються у промисловості для виявлення дефектів продукції та забезпечення відповідності виробів заданим стандартам. Найбільшого поширення вони набули у сферах, де критично важлива точність — зокрема у виробництві оптичних компонентів, електроніки, автомобільних деталей та медичного обладнання. Такі системи дозволяють значно зменшити вплив людського фактора та підвищити стабільність перевірки продукції. Одними з найбільш відомих промислових рішень є системи машинного зору компаній Cognex [2], Keyence [3], Omron [4] та Basler [5].

Рішення компанії Cognex широко використовуються у великих виробничих лініях і відзначаються високою точністю розпізнавання дефектів. Вони підтримують інтеграцію з промисловими роботами та системами автоматизації, а також мають потужні інструменти для аналізу зображень у реальному часі. Основним недоліком таких систем є висока вартість ліцензій та обладнання, а також складність впровадження.

Системи Keyence орієнтовані на максимальну простоту використання у промислових умовах. Вони часто пропонують готові рішення “під ключ”, де значна частина налаштувань виконується через графічний інтерфейс без глибокого програмування. Це робить їх зручними для виробництв, однак обмежує гнучкість налаштування складних алгоритмів аналізу.

Продукти Omron поєднують апаратні та програмні компоненти і активно використовуються у автоматизованих виробничих системах. Вони добре інтегруються у виробничі процеси, але потребують спеціалізованого обладнання та мають значну складність конфігурації.

Рішення Basler більше орієнтовані на роботу з камерами та низькорівневу обробку зображень. Вони часто використовуються як частина більших систем і надають гнучкі інструменти для розробників, однак вимагають самостійного створення логіки аналізу та користувацького інтерфейсу.

Окрему групу аналогів становлять системи, що базуються на методах

штучного інтелекту та глибокого навчання. Такі рішення здатні виявляти складні та нетипові дефекти, які важко формалізувати класичними алгоритмами. Вони використовуються у сучасних виробництвах із великими обсягами даних. Проте їх впровадження потребує значних обчислювальних ресурсів, підготовки великих датасетів та тривалого навчання моделей, що ускладнює використання у невеликих проєктах. Також поширеним інструментарієм для розробки систем комп'ютерного зору є бібліотека OpenCV [6], та інші подібні рішення. Вони надають широкий набір алгоритмів для обробки зображень, включаючи фільтрацію, виділення контурів та аналіз геометрії об'єктів. Водночас ці бібліотеки не є готовими системами контролю якості, оскільки не містять прикладної логіки, інтерфейсу користувача або засобів автоматизації процесу аналізу.

У випадку контролю оптичних виробів, зокрема лінз, додатковою складністю є необхідність виявлення дрібних дефектів поверхні, які можуть бути малопомітними при стандартному освітленні. Багато промислових систем вирішують цю проблему за рахунок спеціалізованого освітлення або багатокамерних конфігурацій, що значно підвищує вартість впровадження та обслуговування.

Аналіз існуючих аналогів показує, що більшість рішень або орієнтовані на великі промислові підприємства з високим бюджетом, або вимагають значних зусиль для самостійної розробки повноцінної системи. Це створює потребу у більш простих, доступних та гнучких рішеннях, які можуть бути адаптовані до задач невеликих виробництв та дослідницьких систем контролю якості.

2 ПОСТАНОВКА ЗАДАЧІ

У результаті аналізу предметної області, сучасних підходів до автоматизованого контролю якості та існуючих програмних рішень було встановлено, що задача автоматичного виявлення дефектів оптичних виробів, зокрема лінз, є актуальною та недостатньо вирішеною у вигляді доступних і простих програмних систем. Більшість існуючих рішень або орієнтовані на великі промислові комплекси, або потребують значних обчислювальних ресурсів і складної інфраструктури, що обмежує їх використання у навчальних, лабораторних або малих виробничих умовах.

У зв'язку з цим виникає необхідність створення програмної системи, яка поєднуватиме засоби обробки зображень, автоматичний аналіз дефектів та зручний інтерфейс взаємодії з користувачем. Така система повинна забезпечувати швидко обробку зображень, стабільну роботу в режимі безперервного моніторингу та можливість формування структурованих звітів для подальшого аналізу результатів. Програмний продукт має бути побудований таким чином, щоб забезпечувати як ручний режим аналізу окремих зображень, так і автоматичний режим обробки великої кількості даних без участі користувача. Важливим аспектом є також забезпечення наочності результатів, що дозволяє оператору швидко оцінити якість виробу.

У процесі розробки програмного забезпечення необхідно реалізувати комплекс функціональних можливостей, що охоплюють усі етапи роботи системи — від отримання зображення до формування фінального висновку про якість виробу. Нижче наведено повний перелік задач, які необхідно вирішити:

- забезпечити можливість завантаження вхідних зображень користувачем із файлової системи, підтримуючи основні графічні формати, зокрема PNG, JPG, JPEG та BMP;
- реалізувати механізм коректного зчитування зображень та їх підготовки до подальшої обробки без втрати якості даних;

- розробити модуль первинної обробки зображень, який включає базові операції комп'ютерного зору, такі як фільтрація шумів, перетворення у відтінки сірого та інверсія зображення;
- забезпечити можливість виділення ключових структурних елементів на зображенні для подальшого аналізу;
- реалізувати алгоритм виявлення лінзи на зображенні та перевірки її наявності у кадрі;
- забезпечити аналіз геометричних характеристик лінзи з метою визначення правильності її форми;
- розробити механізм автоматичного виявлення дефектів поверхні лінзи, включаючи подряпини, плями та інші аномалії;
- реалізувати підрахунок кількості виявлених дефектів та формування числових характеристик результату аналізу;
- забезпечити класифікацію стану об'єкта аналізу на основі отриманих параметрів (наприклад, придатний або дефектний виріб);
- реалізувати механізм формування узагальненого статусу обробки для кожного зображення;
- забезпечити відображення результатів аналізу в інтерфейсі користувача у зрозумілому та наочному вигляді;
- реалізувати багатовіконний або багатовкладковий режим перегляду різних етапів обробки зображення, включаючи оригінальне зображення, проміжні етапи та фінальний результат детекції;
- забезпечити синхронне оновлення всіх візуальних компонентів інтерфейсу після завершення аналізу;

- реалізувати відображення статистичних даних, що включають кількість дефектів, час обробки, статус системи та результати класифікації;
- забезпечити можливість збереження інформації про результати обробки для подальшого використання;
- реалізувати режим одноразової обробки зображення за вибором користувача;
- забезпечити автоматичний режим роботи системи з можливістю безперервного контролю вхідного потоку зображень;
- реалізувати механізм моніторингу директорії, у яку надходять нові зображення для аналізу;
- забезпечити автоматичну обробку кожного нового файлу без необхідності ручного запуску аналізу;
- реалізувати систему автоматичного розподілу оброблених зображень за категоріями залежно від результату аналізу;
- забезпечити переміщення зображень із вхідної директорії до відповідних каталогів “придатні” та “дефектні”;
- реалізувати захист від повторної обробки одного й того ж файлу у межах автоматичного режиму;
- забезпечити коректну обробку ситуацій, коли у вхідній директорії відсутні нові зображення;
- реалізувати можливість стабільної роботи системи у режимі 24/7 без втрати продуктивності;
- забезпечити механізм керування автоматичним режимом (запуск та зупинка моніторингу);

- реалізувати формування PDF-звітів за результатами аналізу окремих зображень;
- забезпечити включення до звіту повної інформації про оброблене зображення, включаючи його назву, дату та час аналізу;
- додати до звіту статистичні дані, такі як кількість дефектів, статус перевірки та час обробки;
- забезпечити візуальне представлення результатів аналізу у звіті у вигляді набору зображень різних етапів обробки;
- реалізувати компоновку звіту таким чином, щоб забезпечити наочність та логічну структуру подання інформації;
- забезпечити автоматичне створення тимчасових файлів для генерації звіту та їх видалення після завершення процесу;
- реалізувати графічний інтерфейс користувача з логічним розміщенням елементів управління;
- забезпечити інформування користувача про поточний стан системи, включаючи процес аналізу, очікування та завершення обробки;
- реалізувати візуальну індикацію результатів класифікації за допомогою кольорових маркерів або текстових повідомлень;
- забезпечити можливість подальшого розширення функціоналу системи без суттєвої зміни архітектури програмного продукту;
- реалізувати модульну структуру системи для спрощення підтримки та модернізації програмного забезпечення;
- забезпечити взаємодію між модулями обробки зображень, інтерфейсу користувача та файлової системи у межах єдиної архітектури.

Сформований перелік задач визначає повний концептуальний опис майбутньої програмної системи автоматизованого контролю якості лінз і є основою для її подальшої реалізації.

3 АРХІТЕКТУРА ТА ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 UML проєктування ПЗ

Проєктування програмного забезпечення системи автоматизованого контролю якості лінз передбачає використання UML-нотації для формалізації основних аспектів поведінки та структури системи. UML (Unified Modeling Language) дозволяє на етапі проєктування чітко визначити взаємодію між компонентами системи, логіку роботи користувацьких сценаріїв, а також внутрішню архітектуру програмних модулів.

Першим етапом UML-проєктування є визначення основних сценаріїв використання системи. В рамках системи можна виділити кілька ключових ролей та взаємодій, де основним актором виступає користувач. Користувач взаємодіє з системою через графічний інтерфейс та може виконувати такі дії, як завантаження зображення для аналізу, запуск автоматичного моніторингу, зупинка моніторингу, перегляд результатів обробки та формування звіту у форматі PDF. Окремим сценарієм є автоматичний режим роботи, у якому система самостійно реагує на появу нових зображень у визначеній директорії без безпосередньої участі користувача.

У контексті діаграми варіантів використання (Use Case Diagram) (див. рис. 3.1) система може бути представлена як набір взаємопов'язаних функцій. Основний актор — користувач — взаємодіє з такими варіантами використання, як «Завантажити зображення», «Запустити автоматичний моніторинг», «Зупинити автоматичний моніторинг», «Переглянути результати аналізу» та «Згенерувати PDF-звіт». У свою чергу, кожен із цих варіантів використання пов'язаний із внутрішніми підсистемами, зокрема модулем обробки зображень, модулем файлового моніторингу та модулем генерації звітів. Наступним важливим етапом є опис поведінки системи за допомогою діаграм послідовностей (Sequence Diagram) (див. рис. 3.2).

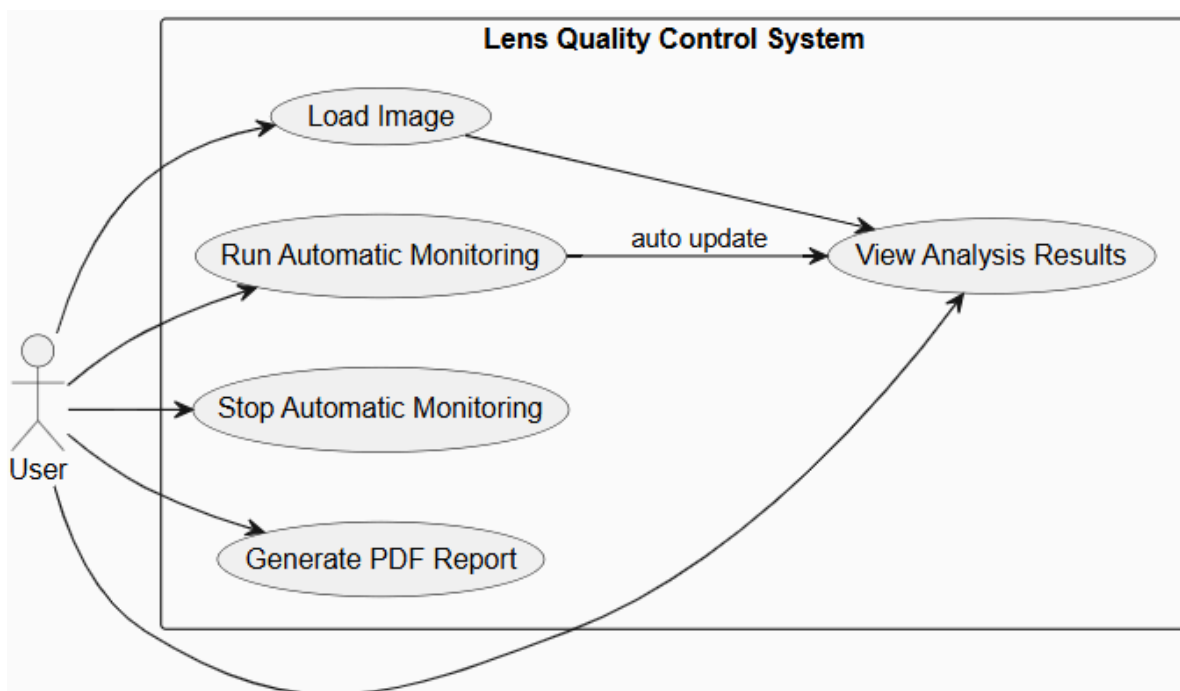


Рисунок 3.1 – Use Case Diagram (рисунок виконано самостійно)

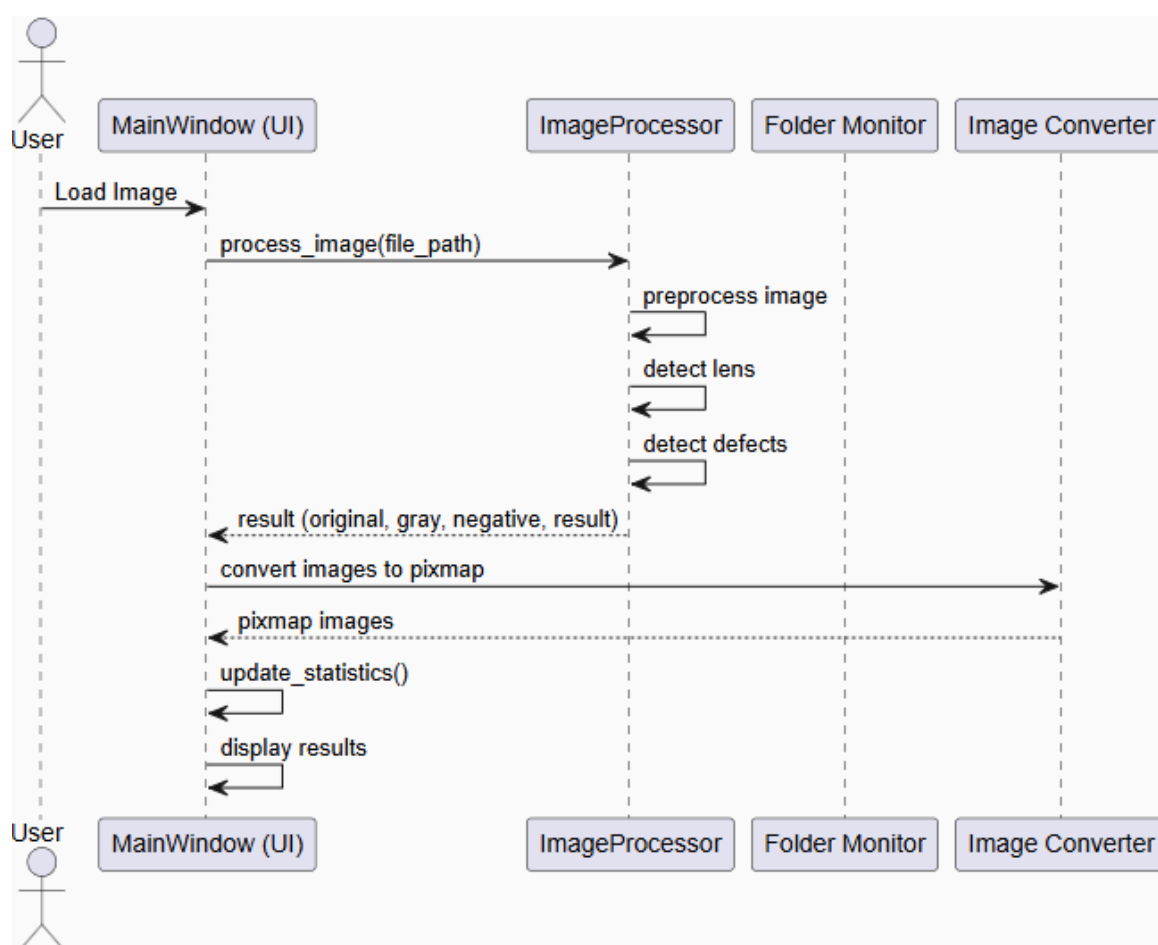


Рисунок 3.2 – Sequence Diagram (рисунок виконано самостійно)

Для сценарію одиничної обробки зображення послідовність дій виглядає наступним чином: користувач ініціює вибір файлу через інтерфейс, система отримує шлях до зображення, після чого передає його до модуля обробки зображень.

Для сценарію одиничної обробки зображення послідовність дій виглядає наступним чином: користувач ініціює вибір файлу через інтерфейс, система отримує шлях до зображення, після чого передає його до модуля обробки зображень. Модуль обробки виконує аналіз зображення, визначає наявність лінзи, виконує пошук дефектів та формує результат. Після цього результат повертається до графічного інтерфейсу, де відображається користувачу у вигляді зображень та текстової інформації. Додатково фіксується час обробки та кількість виявлених дефектів. Для сценарію автоматичного моніторингу послідовність є більш складною. Система у фоновому режимі відстежує директорію входящих файлів. При появі нового зображення модуль моніторингу ініціює його обробку та передає у модуль аналізу. Після отримання результату система приймає рішення про класифікацію зображення: у разі перевищення порогу дефектів файл переміщується до директорії дефектів, в іншому випадку — до директорії оброблених зображень. Паралельно оновлюється інтерфейс користувача, де відображається актуальний статус системи.

Діаграма класів (Class Diagram) (див. рис. 3.3) відображає структуру основних компонентів системи. Центральним класом є клас головного вікна, який відповідає за взаємодію з користувачем та координацію роботи всієї системи. Він містить методи для завантаження зображень, запуску моніторингу, оновлення інтерфейсу та виклику функцій генерації звітів. Окремо виділяється клас обробки зображень, який інкапсулює алгоритми комп'ютерного зору, включаючи попередню обробку, виявлення лінзи та аналіз дефектів. Також існує модуль моніторингу директорій, який відповідає за відстеження файлової системи та ініціалізацію обробки нових файлів.

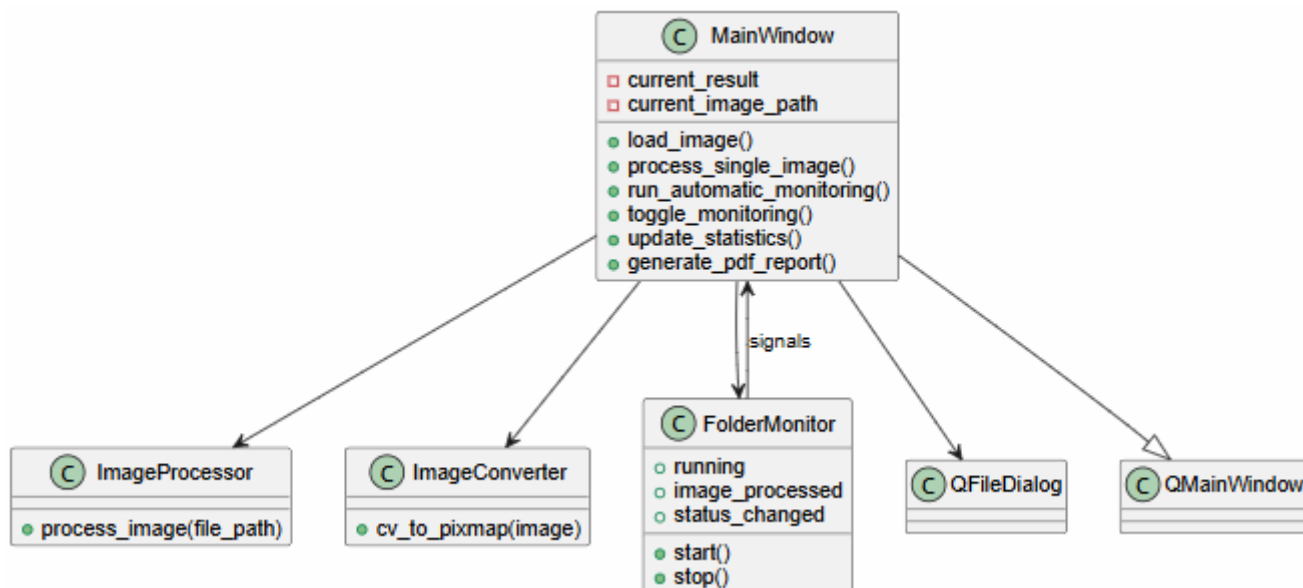


Рисунок 3.3 – Class Diagram (рисунок виконано самостійно)

Додатково присутній модуль конвертації зображень для відображення результатів у графічному інтерфейсі та модуль генерації звітів, який формує PDF-документи на основі отриманих результатів.

3.2 Проектування архітектури програмного забезпечення

Проектування архітектури програмного забезпечення є одним із ключових етапів розробки системи автоматизованого контролю якості лінз, оскільки саме архітектура визначає структуру програмного продукту, принципи взаємодії між його компонентами та загальну організацію обробки даних. Від правильності побудови архітектури залежить стабільність роботи системи, можливість її масштабування, підтримки та подальшого вдосконалення. Основною задачею архітектурного проектування було створення програмної системи, яка забезпечує автоматичну обробку зображень, підтримує безперервний моніторинг вхідних даних та дозволяє гнучко керувати процесом аналізу. При цьому система повинна залишатися достатньо простою для використання та підтримки, а також не потребувати складної інфраструктури або спеціалізованого обладнання. В основу архітектури було покладено модульний підхід. Такий підхід передбачає поділ програмної системи на окремі логічні компоненти, кожен із яких виконує

визначений набір функцій. Це дозволяє ізолювати окремі частини системи одна від одної, спрощує тестування та забезпечує можливість незалежного вдосконалення окремих модулів без необхідності повного перепроєктування всієї системи.

Архітектура системи включає кілька основних функціональних модулів: модуль графічного інтерфейсу, модуль обробки зображень, модуль моніторингу директорій, модуль генерації звітів та допоміжні модулі для конвертації та відображення зображень. Кожен із цих модулів виконує окрему роль у загальному процесі роботи системи.

Центральним компонентом архітектури є модуль керування, який реалізований у вигляді головного вікна програми. Даний модуль координує взаємодію між усіма іншими компонентами системи. Саме через нього здійснюється запуск процесів аналізу, оновлення результатів, передача даних між модулями та контроль за станом автоматичного моніторингу. Головне вікно виступає своєрідним центром керування всією програмною системою. Модуль обробки зображень є основою функціональності системи. Його архітектура побудована навколо послідовного виконання етапів цифрової обробки зображення. Після отримання вхідного файлу система виконує завантаження зображення у пам'ять та його попередню підготовку. Далі здійснюється виділення області лінзи, аналіз геометричних параметрів та пошук дефектів поверхні. Для підвищення ефективності аналізу використовується декілька варіантів обробки одного й того самого зображення. Окрім основного режиму аналізу, система формує додаткові представлення зображення, зокрема покращене градаційне зображення та негатив. Такий підхід дозволяє покращити видимість окремих типів дефектів та підвищити загальну точність аналізу. Архітектура модуля обробки побудована таким чином, щоб результат аналізу повертався у вигляді структурованого набору даних. До нього входять оригінальне зображення, результати різних типів обробки, кількість знайдених дефектів, текстовий статус аналізу та службова інформація. Це дозволяє іншим модулям системи працювати із результатами незалежно від внутрішньої реалізації алгоритмів аналізу.

Важливою складовою архітектури є модуль автоматичного моніторингу директорій. Його задача полягає у постійному відстеженні вмісту папки incoming та автоматичному запуску аналізу при появі нових файлів. Архітектурно цей модуль реалізований окремо від основного інтерфейсу, що дозволяє системі працювати у фоновому режимі без блокування користувацьких дій. Принцип роботи модуля моніторингу базується на циклічній перевірці вмісту директорії через визначені проміжки часу. Після виявлення нового зображення модуль передає шлях до файлу у систему обробки. Після завершення аналізу система автоматично класифікує результат та переміщує файл до відповідної директорії. Якщо кількість дефектів перевищує встановлений поріг, файл переноситься до папки defects, інакше — до папки processed. Окрему увагу при проектуванні архітектури було приділено стабільності роботи системи у режимі безперервного моніторингу. Оскільки програма повинна працювати тривалий час у режимі 24/7, необхідно було забезпечити коректну обробку помилок, уникнення повторної обробки файлів та стійкість до змін у файловій системі. Саме тому логіка моніторингу була відокремлена від логіки графічного інтерфейсу та основної обробки зображень.

Ще одним важливим компонентом архітектури є модуль генерації PDF-звітів. Його задача полягає у створенні структурованого документа на основі результатів аналізу. Архітектурно цей модуль працює незалежно від процесу детекції та отримує вже готові результати обробки. Під час формування звіту система використовує інформацію про зображення, кількість дефектів, дату та час аналізу, а також результати всіх режимів обробки.

Для забезпечення сумісності між компонентами використовується єдина структура даних результату аналізу. Це дозволяє передавати результати між модулями без додаткового перетворення даних та спрощує інтеграцію нових функцій у майбутньому. Архітектура системи також враховує принцип розділення відповідальностей. Кожен модуль відповідає лише за власний функціонал та не містить надлишкової логіки. Наприклад, модуль обробки зображень не виконує операцій із файловою системою, а модуль моніторингу не містить алгоритмів

аналізу дефектів. Для реалізації програмної системи було обрано мову програмування Python [7]. Даний вибір обумовлений наявністю великої кількості бібліотек для комп'ютерного зору, роботи з графічними інтерфейсами та автоматизації обробки даних. Зокрема, для обробки зображень використовується бібліотека OpenCV, для реалізації графічного інтерфейсу — PySide6 [8], а для генерації звітів — ReportLab [9]. Архітектура програмного забезпечення побудована таким чином, щоб забезпечити можливість подальшого розширення функціоналу. У майбутньому система може бути доповнена підтримкою нових алгоритмів детекції, інтеграцією із промисловими камерами, базами даних або системами статистичного аналізу результатів контролю якості.

3.3 Створення UI/UX

Під час проектування програмної системи контролю якості оптичних лінз особлива увага приділялася створенню зручного та зрозумілого користувацького інтерфейсу. Оскільки система орієнтована на використання у виробничому середовищі, інтерфейс повинен забезпечувати швидке сприйняття інформації, мінімальну кількість зайвих дій оператора та можливість безперервної роботи протягом тривалого часу. Саме тому при створенні UI/UX було обрано підхід, заснований на простоті, візуальній інформативності та мінімізації когнітивного навантаження на користувача. Основною задачею інтерфейсу є забезпечення максимально швидкого доступу до результатів аналізу лінз. Оператор повинен мати можливість миттєво визначити стан системи, переглянути результати детекції дефектів та отримати всю необхідну технічну інформацію без переходу між великою кількістю меню або додаткових вікон. У зв'язку з цим було реалізовано одновіконний інтерфейс, у якому всі ключові елементи розташовані на головному екрані. При створенні графічного інтерфейсу використовувалася бібліотека PySide6, яка дозволяє реалізовувати сучасні десктопні застосунки з використанням компонентів Qt [9]. Даний інструментарій забезпечує високу швидкодію, підтримку багатоплатформеності та широкі можливості налаштування зовнішнього вигляду елементів інтерфейсу.

Головне вікно системи (див. рис. 3.4) складається з декількох логічних областей.

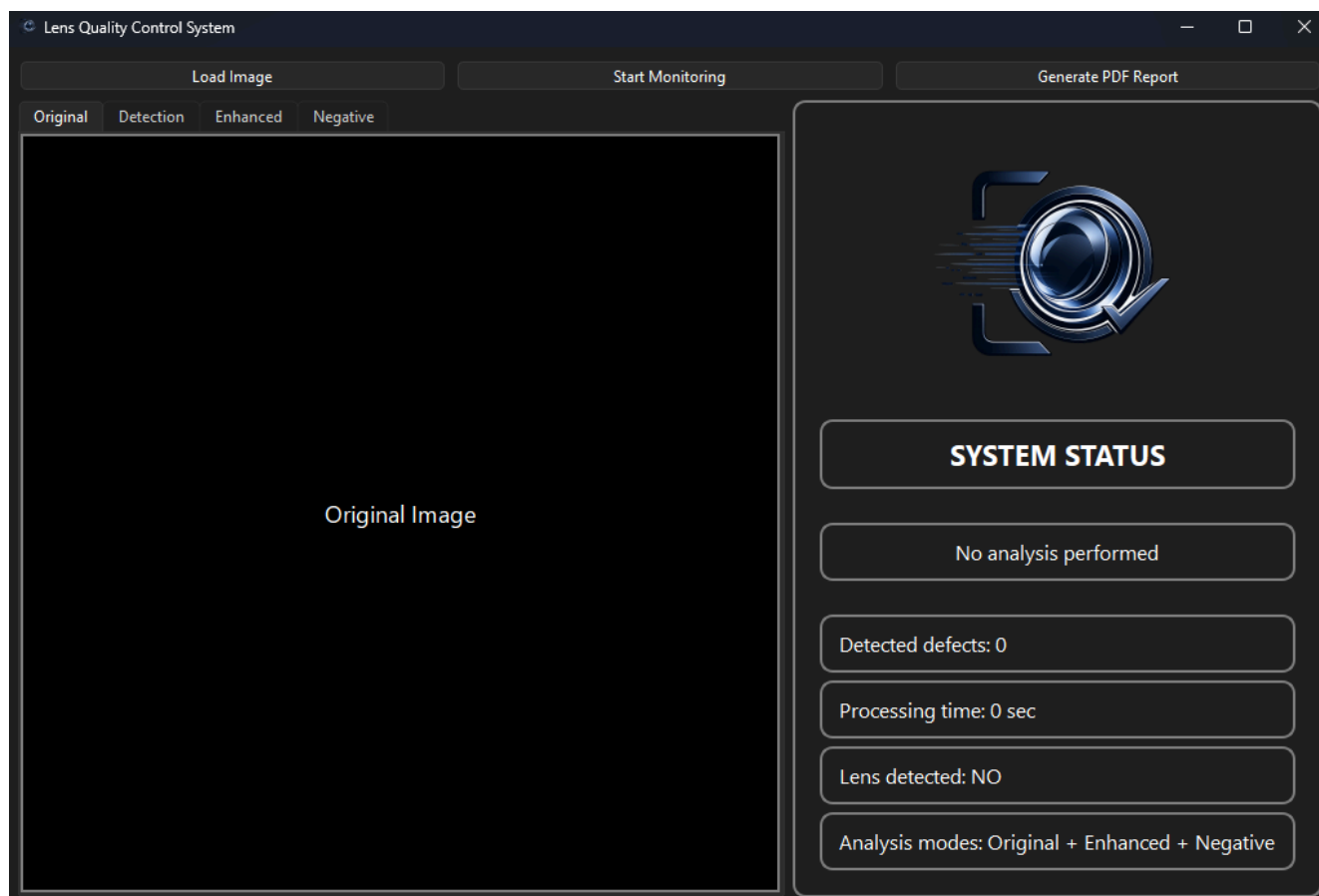


Рисунок 3.4 – Головне вікно системи (рисунок виконано самостійно)

У верхній частині розташована панель керування, яка містить основні кнопки взаємодії з системою. До них належать кнопки завантаження зображення, запуску або зупинки автоматичного моніторингу та генерації PDF-звіту. Таке розташування було обране через те, що верхня частина інтерфейсу є найбільш помітною для користувача та забезпечує швидкий доступ до основних функцій системи. Центральну частину вікна займає область відображення зображень. Для забезпечення зручності аналізу було реалізовано систему вкладок, кожна з яких відповідає окремому режиму відображення результатів обробки. Користувач може переглядати:

- оригінальне зображення лінзи;
- результат основної детекції дефектів;

- покращене grayscale-зображення;
- негативне зображення для альтернативного аналізу поверхні.

Поділ результатів на окремі вкладки дозволяє уникнути перевантаження інтерфейсу великою кількістю графічної інформації одночасно. Крім цього, оператор може швидко перемикатися між режимами аналізу та оцінювати ефективність різних способів обробки зображення. Для відображення зображень використовуються спеціальні області QLabel, у яких реалізовано автоматичне масштабування зображення зі збереженням пропорцій. Це дозволяє коректно відображати фотографії різних розмірів та роздільної здатності без втрати візуальної якості. Додатково було використано темне фонове оформлення області перегляду, що покращує контрастність та полегшує візуальне сприйняття результатів детекції. Праворуч від області зображень розташована інформаційна панель стану системи. Вона містить основні параметри поточного аналізу, серед яких:

- статус роботи системи;
- кількість виявлених дефектів;
- час обробки зображення;
- інформація про успішність виявлення лінзи;
- перелік режимів аналізу.

Дана панель виконує роль інформаційного центру системи та дозволяє оператору швидко оцінити результати перевірки без необхідності детального перегляду всіх зображень. Увагу було приділено кольоровому оформленню статусів. Для позитивних результатів використовуються зелені кольори, а для повідомлень про помилки або дефекти — червоні. Такий підхід забезпечує миттєве візуальне сприйняття критично важливої інформації.

Однією з важливих складових UX стала реалізація автоматичного режиму моніторингу папки incoming. У цьому режимі система постійно працює у

фоновому режимі та автоматично реагує на появу нових зображень. Для користувача цей процес виглядає максимально просто: достатньо помістити файл у відповідну папку, після чого система самостійно виконує аналіз, оновлює інтерфейс та переміщує файл до відповідної директорії. Завдяки цьому значно зменшується кількість ручних операцій та підвищується загальна ефективність роботи. При проектуванні UX також враховувалися особливості виробничого середовища. Оскільки система може працювати безперервно протягом тривалого часу, інтерфейс був спроектований таким чином, щоб уникати надлишкових анімацій, спливаючих вікон та інших елементів, які можуть відволікати оператора або ускладнювати контроль процесу. Водночас система надає достатню кількість повідомлень про свій стан, що дозволяє оператору своєчасно реагувати на можливі помилки або нестандартні ситуації.

Додатково було реалізовано функцію генерації PDF-звітів. Інтерфейс дозволяє створювати звіт натисканням однієї кнопки без необхідності введення додаткових параметрів. Такий підхід мінімізує час формування документації та спрощує роботу персоналу. Згенерований звіт містить усю ключову інформацію про виконаний аналіз, включаючи дату, назву файлу, результати і час детекції та графічні матеріали.

У процесі проектування інтерфейсу основна увага приділялася забезпеченню балансу між функціональністю та простотою використання. Інтерфейс не перевантажений другорядними елементами, а всі ключові функції доступні безпосередньо з головного вікна. Це дозволяє забезпечити швидке навчання персоналу та ефективну роботу навіть для користувачів без глибоких технічних знань.

4 ОПИС ПРИЙНЯТИХ ПРОГРАМНИХ РІШЕНЬ

4.1 Обробка зображення

Під час розробки програмної системи автоматичного контролю якості оптичних лінз було прийнято ряд програмних рішень, спрямованих на забезпечення високої швидкості роботи, стабільності функціонування та достатньої точності виявлення дефектів поверхні. Основна увага приділялася створенню системи, здатної працювати у напівавтоматичному та автоматичному режимах без необхідності постійного втручання оператора. Ключовою частиною програмного забезпечення став модуль цифрової обробки зображень, реалізований у файлі `image_processor.py`. Саме цей модуль відповідає за завантаження зображень, підготовку даних до аналізу, визначення меж лінзи, формування масок, виявлення дефектів та генерацію фінального результату детекції.

Одним із перших прийнятих рішень стала реалізація окремого механізму завантаження зображень із підтримкою Unicode-символів у шляхах до файлів. Стандартні методи OpenCV у деяких випадках некоректно працюють із кириличними символами або спеціальними знаками у назвах директорій та файлів. Для усунення даної проблеми було реалізовано окрему функцію `load_image_unicode()`, яка виконує зчитування файлу у вигляді масиву байтів за допомогою NumPy [11], після чого декодує його у формат OpenCV.

```
@staticmethod
def load_image_unicode(image_path):

    data = np.fromfile(
        image_path,
        dtype=np.uint8
    )

    return cv2.imdecode(
        data,
        cv2.IMREAD_COLOR
    )
```

Дане рішення дозволило забезпечити стабільну роботу системи незалежно від мови операційної системи або назв файлів користувача. Це особливо важливо

для промислових умов, де структура директорій може формуватися автоматично та містити різні локалізації.

Наступним важливим етапом стала реалізація механізму покращення контрастності зображення. Під час аналізу фотографій оптичних лінз значну проблему становлять неоднорідне освітлення, низький контраст дрібних подряпин та локальні перепади яскравості. Для вирішення цієї задачі було використано алгоритм CLAHE (Contrast Limited Adaptive Histogram Equalization) [12], який дозволяє локально підсилювати контраст без надмірного посилення шумів.

```
@staticmethod
def enhance_contrast(gray):

    clahe = cv2.createCLAHE(
        clipLimit=3.0,
        tileGridSize=(8, 8)
    )

    return clahe.apply(gray)
```

Використання CLAHE дозволило значно покращити видимість дрібних дефектів поверхні, особливо у випадках, коли стандартне grayscale-зображення не забезпечувало достатньої деталізації (див. рис. 4.1).

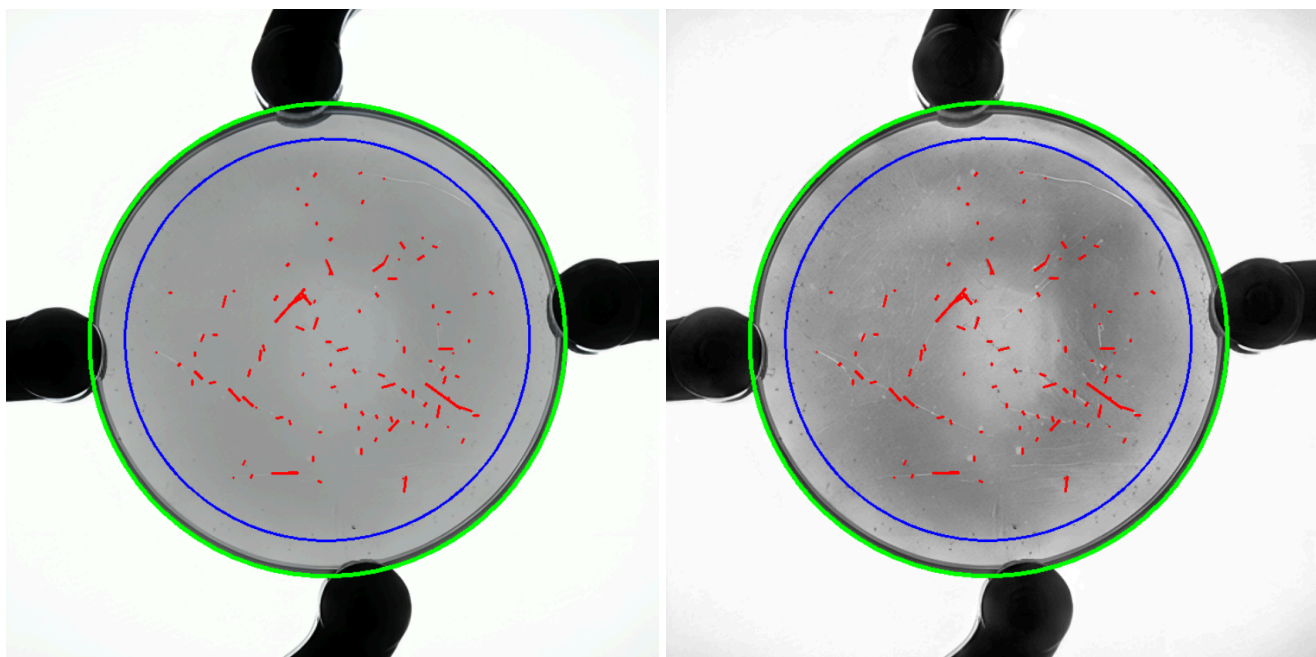


Рисунок 4.1 – Порівняння оригінального та деталізованого зображень (рисунок виконано самостійно)

Дане рішення також позитивно вплинуло на подальшу роботу алгоритмів виявлення границь та ліній.

Одним із найбільш складних та важливих програмних рішень стала реалізація алгоритму визначення меж лінзи. У класичних системах комп'ютерного зору для цього часто використовуються методи порогової сегментації або Hough Circle Transform [13]. Проте через неоднорідність освітлення, різні типи поверхонь та можливу присутність шумів такі методи не забезпечували стабільного результату. У зв'язку з цим було прийнято рішення реалізувати власний алгоритм радіального аналізу зображення. Суть алгоритму полягає у скануванні зображення від центру до периферії по великій кількості променів. Для кожного напрямку виконується пошук точки з максимальним градієнтом яскравості, яка потенційно відповідає межі лінзи. Даний підхід дозволяє достатньо точно визначати контури навіть при наявності часткових шумів або локальних дефектів.

```
for angle in np.linspace(
    0,
    2 * np.pi,
    ImageProcessor.RAY_COUNT
):

    prev = int(gray[cy, cx])

    best_gradient = 0
    best_point = None

    for r in range(20, max_radius):

        x = int(cx + r * math.cos(angle))
        y = int(cy + r * math.sin(angle))

        if x < 0 or x >= w or y < 0 or y >= h:
            break

        value = int(gray[y, x])

        gradient = abs(value - prev)

        if gradient > best_gradient:
            best_gradient = gradient
            best_point = (x, y)

    prev = value
```

```

if (
    best_point is not None and
    best_gradient > ImageProcessor.EDGE_THRESHOLD
):
    points.append(best_point)

```

Перевагою такого підходу стала його стійкість до локальних дефектів поверхні та змін освітлення. Крім цього, алгоритм не потребує попереднього жорсткого порогового перетворення зображення, що дозволяє працювати із ширшим спектром вхідних фотографій.

Після визначення межових точок було реалізовано додатковий механізм фільтрації аномальних значень. Під час аналізу окремі точки могли значно відхилятися від реальної геометрії лінзи через шуми, відблиски або дефекти. Для усунення таких ситуацій використовується метод фільтрації на основі медіанного відхилення. Алгоритм обчислює відстань кожної точки до умовного центру та порівнює її з медіанним значенням. Точки, відхилення яких перевищує допустимий поріг, виключаються з подальшого аналізу. Після фільтрації межових точок система переходить до побудови еліпса, який описує форму лінзи. Використання еліпса замість кола дозволяє коректно працювати навіть у випадках неідеального положення камери або перспективних спотворень.

Окремої уваги заслуговує механізм формування масок аналізу. Для зменшення кількості помилкових спрацювань було реалізовано повну маску лінзи, та окрему зону аналізу пошкоджень (див. рис. 4.2). Такий підхід дозволив виключити краї лінзи із процесу детекції, оскільки саме на периферії найчастіше виникають артефакти освітлення та помилкові контури. Для побудови внутрішньої області аналізу використовується наступна реалізація:

```

@staticmethod
def create_inner_safe_mask(
    shape,
    ellipse,
    shrink=0.85
):
    mask = np.zeros(
        shape[:2],
        dtype=np.uint8
    )

```

```

)

(cx, cy), (ax, ay), angle = ellipse

safe = (
    (cx, cy),
    (ax * shrink, ay * shrink),
    angle
)

cv2.ellipse(
    mask,
    safe,
    255,
    -1
)

return mask

```

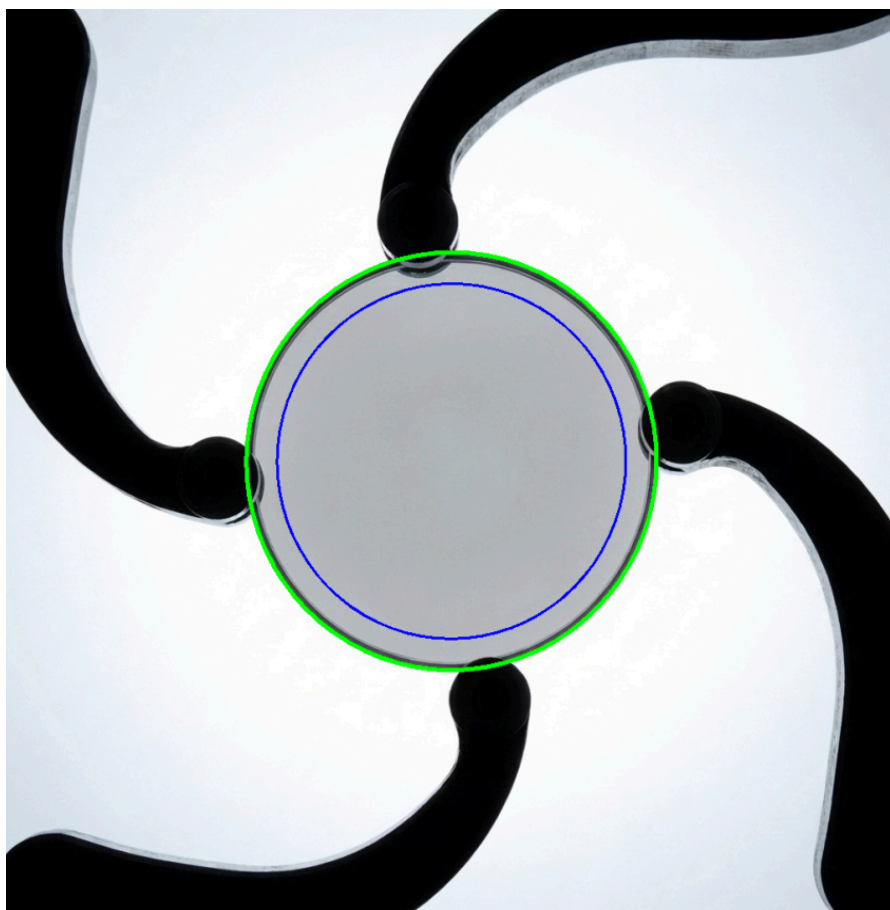


Рисунок 4.2 – Повна маска лінзи (зелений) та зона аналізу пошкоджень (синій)
(рисунок виконано самостійно)

Важливим програмним рішенням також стало використання нормалізації фону перед детекцією дефектів. Через неоднорідне освітлення поверхні на

зображенні можуть виникати плавні перепади яскравості, які негативно впливають на алгоритми пошуку подряпин. Для усунення цієї проблеми застосовується Gaussian Blur великого радіуса, після чого виконується обчислення різниці між оригіналом та розмитим фоном. Таким чином система фактично виділяє локальні зміни поверхні та пригнічує великомасштабні зміни освітлення. Після нормалізації виконується пошук дефектів за допомогою алгоритму Canny Edge Detector [14] та морфологічних операцій. Додатково було реалізовано окремий режим аналізу негативного зображення. Практичні експерименти показали, що деякі типи подряпин стають значно помітнішими саме після інверсії яскравості (див. рис. 4.3).

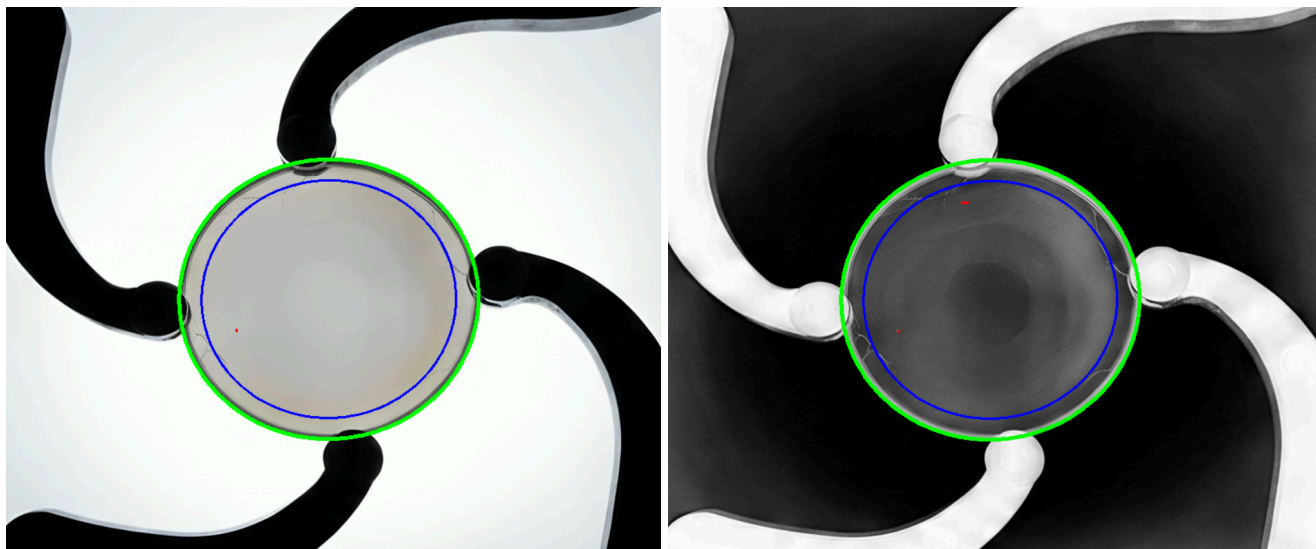


Рисунок 4.3 – Порівняння результатів детекції оригінального та негативного зображень (рисунок виконано самостійно)

Для підвищення точності визначення дефектів використовується алгоритм HoughLinesP, який дозволяє знаходити лінійні структури на поверхні лінзи. Оскільки одна і та сама подряпина може бути визначена декілька разів, додатково реалізовано перевірку на дублювання ліній.

```
for ux1, uy1, ux2, uy2 in used_lines:
    dist = math.sqrt(
        (x1 - ux1) ** 2 +
        (y1 - uy1) ** 2
```

```

    )

    if dist < 15:
        duplicate = True
        break

    if duplicate:
        continue

```

Дане рішення дозволило значно зменшити кількість повторних детекцій одного й того самого дефекту та зробити результати аналізу більш стабільними.

4.2 Автоматичний моніторинг

Окремою складовою програмної системи став модуль автоматичного моніторингу папки `incoming`, реалізований у файлі `folder_monitor.py`. Для забезпечення безперервної роботи системи було прийнято рішення реалізувати моніторинг у окремому потоці за допомогою класу `QThread`. Основна логіка моніторингу полягає у циклічному опитуванні директорії та автоматичній обробці нових файлів. Після завершення аналізу система самостійно переміщує файли до директорій `processed` або `defects`. Ключова частина алгоритму виглядає наступним чином:

```

while self.running:

    files = [

        f for f in os.listdir(
            self.incoming_folder
        )

        if f.lower().endswith(
            (
                ".png",
                ".jpg",
                ".jpeg",
                ".bmp"
            )
        )
    ]

    for file_name in files:

        if file_name in processed_files:
            continue

```

Використання окремого потоку дозволило уникнути блокування графічного інтерфейсу під час безперервної роботи системи. Користувач може взаємодіяти із програмою навіть у момент автоматичної обробки нових зображень. Додатково було реалізовано механізм передачі сигналів між потоком моніторингу та головним GUI-вікном. Для цього використовуються сигнали `image_processed` та `status_changed`. Завдяки цьому інтерфейс автоматично оновлюється після завершення кожного аналізу.

5 АНАЛІЗ ОТРИМАНИХ РЕЗУЛЬТАТІВ

У процесі виконання курсового проєкту було реалізовано програмну систему автоматизованого контролю якості оптичних лінз на основі методів комп'ютерного зору та цифрової обробки зображень. Результатом роботи стала настільна програмна система, яка забезпечує завантаження та аналіз зображень лінз, виявлення дефектів поверхні, автоматичне сортування оброблених файлів, формування PDF-звітів та підтримку режиму безперервного моніторингу директорії з новими зображеннями. Під час розробки було досягнуто основної мети проєкту — створення програмного забезпечення, здатного автоматизувати процес первинного контролю поверхні лінз без необхідності постійного втручання оператора. Система успішно виконує обробку графічних даних у реальному часі, що дозволяє використовувати її в умовах безперервного виробничого процесу. Реалізований алгоритм аналізу зображень дозволив забезпечити стабільне визначення меж лінзи, побудову еліптичної моделі поверхні та локалізацію потенційних дефектів. Для підвищення точності аналізу було застосовано декілька підходів до попередньої обробки зображення, зокрема підсилення контрасту методом CLAHE, нормалізацію фону, аналіз негативного зображення та виділення контурів за допомогою алгоритму Canny. Комбінація цих методів дозволила підвищити чутливість системи до дрібних подряпин і поверхневих артефактів. Особливу увагу було приділено автоматизації роботи системи. Реалізований механізм моніторингу папки забезпечує постійне відстеження появи нових зображень у директорії incoming. Після виявлення нового файлу система автоматично запускає процедуру аналізу, визначає наявність дефектів та переміщує файл до відповідної папки processed або defects. Такий підхід дозволяє використовувати програму у режимі 24/7 без необхідності ручного запуску кожної операції. У ході тестування було встановлено, що система коректно працює як із якісними зображеннями, так і з фотографіями, які містять різний рівень шумів, нерівномірне освітлення або незначні зміщення об'єкта в кадрі. Використання масок безпечної області дозволило мінімізувати кількість хибних спрацьовувань

поблизу меж лінзи, де часто виникають артефакти через зміну яскравості та геометричні спотворення. Окремим важливим результатом стало створення підсистеми генерації PDF-звітів. Звіт містить повну інформацію щодо виконаного аналізу, включаючи назву зображення, дату та час обробки, кількість виявлених дефектів, статус лінзи та результати всіх етапів детекції. Формування звітної документації значно спрощує процес збереження результатів контролю та забезпечує можливість подальшого аналізу отриманих даних. Розроблений графічний інтерфейс користувача забезпечує просту та зрозумілу взаємодію із системою. Інтерфейс дозволяє виконувати ручне завантаження зображень, запускати та зупиняти автоматичний моніторинг, переглядати результати детекції у різних режимах та формувати звіти. Використання бібліотеки PySide6 дозволило створити сучасний інтерфейс із вкладками для візуалізації окремих типів обробки зображення.

Отримані результати підтверджують, що поставлені завдання курсового проєкту були успішно виконані. Створена система відповідає основним функціональним вимогам, забезпечує автоматизований контроль якості лінз та демонструє стабільну роботу в умовах тривалого використання. Разом із тим проведений аналіз показав можливість подальшого вдосконалення програмного забезпечення. У перспективі систему можна розширити шляхом використання алгоритмів машинного навчання для класифікації типів дефектів, інтеграції підтримки відеопотоку з промислових камер, додавання бази даних для збереження історії перевірок та реалізації статистичного аналізу результатів контролю. Також можливим напрямом розвитку є оптимізація продуктивності для роботи з великими потоками зображень у промислових умовах.

ВИСНОВКИ

У результаті виконання курсового проєкту було розроблено програмну систему автоматизованого контролю якості оптичних лінз із використанням методів комп'ютерного зору та цифрової обробки зображень. У ході роботи було проведено аналіз предметної області, визначено основні проблеми існуючих підходів до контролю якості поверхні лінз та досліджено сучасні програмні рішення, що використовуються у сфері автоматизованого візуального контролю. На основі проведеного аналізу було сформовано вимоги до майбутньої системи та спроектовано архітектуру програмного забезпечення. Для реалізації програмної системи було використано мову програмування Python, бібліотеки OpenCV та NumPy для обробки зображень, а також PySide6 для створення графічного інтерфейсу користувача. У процесі розробки було реалізовано алгоритми визначення меж лінзи, побудови еліптичної моделі поверхні та виявлення дефектів на основі аналізу контурів і градієнтів зображення. Також було реалізовано механізм автоматичного моніторингу папки із зображеннями, що дозволяє системі працювати у безперервному режимі та автоматично виконувати обробку нових файлів. Додатково в програмі реалізовано функціонал формування PDF-звітів, які містять результати аналізу, службову інформацію та зображення всіх етапів обробки. Розроблений графічний інтерфейс дозволяє виконувати як ручний, так і автоматичний режим роботи. Результати виконаної роботи демонструють ефективність використання методів комп'ютерного зору для автоматизації процесів контролю якості оптичних виробів. Розроблена система може бути використана як основа для подальшого розвитку промислових рішень у сфері автоматизованого технічного контролю.

Посилання на GitHub з папкою програми:
https://github.com/NureZhylienkovAndrii/KKP/tree/main/2026_%D0%91_%D0%9A%D0%9A%D0%9F_%D0%9F%D0%97%D0%9F%D0%86-23-5_%D0%96%D0%B8%D0%BB%D1%94%D0%BD%D0%BA%D0%BE%D0%B2_%D0%90_%D0%9E

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

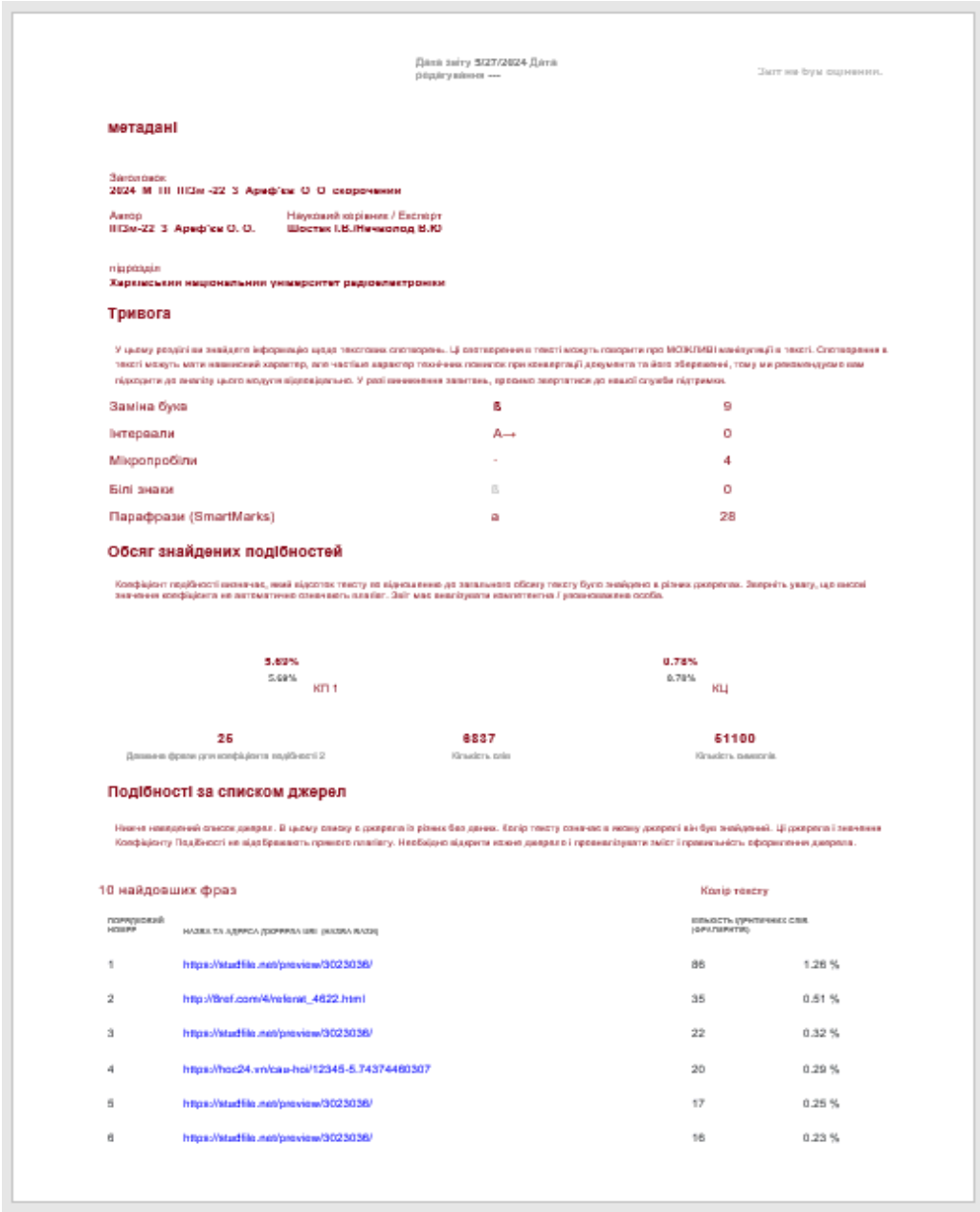
1. Industry 4.0: the Fourth Industrial Revolution – European Parliament [Електронний ресурс] – URL: [https://www.europarl.europa.eu/thinktank/en/document/EPRS_BRI\(2015\)568337](https://www.europarl.europa.eu/thinktank/en/document/EPRS_BRI(2015)568337) (дата звернення: 27.05.2026)
2. Cognex [Електронний ресурс] – URL: <https://www.cognex.com/> (дата звернення: 27.05.2026)
3. Keyence Machine Vision Systems [Електронний ресурс] – URL: <https://www.keyence.com/products/vision/vision-sys/> (дата звернення: 27.05.2026)
4. Omron Machine Automation and Vision Systems [Електронний ресурс] – URL: <https://industrial.omron.eu/en/home> (дата звернення: 27.05.2026)
5. Basler Computer Vision Solutions [Електронний ресурс] – URL: <https://www.baslerweb.com/en/> (дата звернення: 27.05.2026)
6. OpenCV Documentation [Електронний ресурс] – URL: <https://docs.opencv.org/> (дата звернення: 27.05.2026)
7. Python Documentation [Електронний ресурс] – URL: <https://docs.python.org/3/> (дата звернення: 27.05.2026)
8. PySide6 Documentation [Електронний ресурс] – URL: <https://doc.qt.io/qtforpython-6/> (дата звернення: 27.05.2026)
9. ReportLab Documentation [Електронний ресурс] – URL: <https://www.reportlab.com/> (дата звернення: 27.05.2026)
10. Qt Documentation [Електронний ресурс] – URL: <https://doc.qt.io/> (дата звернення: 27.05.2026)
11. NumPy Documentation [Електронний ресурс] – URL: <https://numpy.org/doc/> (дата звернення: 27.05.2026)
12. CLAHE (Contrast Limited Adaptive Histogram Equalization) [Електронний ресурс] – URL: https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html (дата звернення: 27.05.2026)

13. Szeliski R. Computer Vision: Algorithms and Applications. – 2-ге вид. – Cham: Springer, 2022. – 979 с.

14. Canny Edge Detection – OpenCV [Электронный ресурс] – URL: https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html (дата звернення: 27.05.2026)

ДОДАТОК А

Звіт результатів перевірки на унікальність тексту в базі ХНУРЕ



7	https://studfile.net/preview/3023036/	15	0.22 %
8	https://studfile.net/preview/3023036/	12	0.18 %
9	https://studfile.net/preview/3023035/page-13/	10	0.15 %
10	Розвиток системи управління ресурсним потенціалом аграрних підприємств. Болоник Дмитро Володимирович	9	0.13 %

з бази даних RefBooks (0.10 %)

позначка назва	заголовок	кількість цитувань (оригіналів)	
джерело: https://elibrary.org/			
1	Quantile Regression Modeling via Location and Scale Modifiers of Normal Distributions Haim Y. Bar, Martin T. Wells, James G. Booth	7 (1)	0.10 %

з бази даних УкрІНТЕІ (0.56 %)

позначка назва	заголовок	кількість цитувань (оригіналів)	
1	Розвиток системи управління ресурсним потенціалом аграрних підприємств. Болоник Дмитро Володимирович	25 (3)	0.37 %
2	Трансформація взаємодієносні споживачі і виробники у глобальних логістичних мережах Курант Тетяна Ігорівна	13 (2)	0.19 %

з домашньої бази даних (0.00 %)

позначка назва	заголовок	кількість цитувань (оригіналів)	
-------------------	-----------	------------------------------------	--

з програми обміну базами даних (0.34 %)

позначка назва	заголовок	кількість цитувань (оригіналів)	
1	ПРОГНОЗУВАННЯ ІНДИВІДУАЛЬНИХ НАВАНТАЖЕНЬ ХОДЬБИ НА ОСНОВІ ІНТЕГРОВАНОЇ МОДЕЛІ АВТОРЕГРЕСІЇ - ЗМІННОГО СЕРЕДНЬОГО 12/3/2021 Plyazovskiy State Technical University (PSTU)	14 (2)	0.20 %
2	Kovalyova_diplom_Lazarik_computer science_2021.pdf 5/28/2021 V. N. Kuznetsov Kharkiv National University (KNU) (Факультет комп'ютерних наук - кафедра штучного інтелекту та програмного забезпечення)	9 (1)	0.13 %

з Інтернету (4.70 %)

позначка назва	зображення 	кількість цитувань (оригіналів)	
1	https://studfile.net/preview/3023036/	203 (11)	2.97 %
2	https://sc24.vn/cas-hoi/12345-5.74374460307	37 (3)	0.54 %
3	http://bref.com/4/relaw/_4622.html	35 (1)	0.51 %
4	https://ela.kpi.ua/bitstream/123456789/23559/1/Tarasjuk_magistr.pdf	16 (2)	0.23 %
5	https://studfile.net/preview/3023035/page-13/	10 (1)	0.15 %

6	https://conferences.vnu.edu.ua/public/files/1/Vntu_2016_netpub.pdf	9 (1)	0.13 %
7	http://antibiotik.com/file.html?work_id=516971	6 (1)	0.09 %
8	http://www.ncbi.nlm.nih.gov/pubmed/health/PMH0046231/	5 (1)	0.07 %

Список принятых фрагментів (немає принятых фрагментів)

порядковий номер зміст кількість фрагментів: слів (змішаних)

ДОДАТОК Б

Слайди презентації



Програмна система контролю якості виробництва лінз за зображенням їх поверхні

Комплексний курсовий проєкт

COMPUTER VISION

Студент:

Жиленков Андрій Олександрович

Група: ПЗПІ-23-5

INDUSTRY 4.0

OPENCV · PYTHON · PYSIDE6

Науковий керівник:

доц., к.т.н. Работягов Андрій Валентинович

2026

1

Аналіз предметної галузі

Актуальність та роль лінз у технологіях

Сучасне виробництво вимагає надзвичайно високої точності та стабільного контролю якості. Особливо це критично у сфері оптичних виробів: навіть **мікродефекти** поверхні лінзи суттєво впливають на якість зображення, точність фокусування та оптичні спотворення кінцевого пристрою.

Оптичні лінзи є ключовими компонентами широкого спектру сучасних технологій:

- Фото- та відеотехніка, смартфони та камери спостереження
- Медичні прилади та діагностичне обладнання
- Мікроскопи, телескопи та наукові інструменти
- Лазерні системи та промислові установки

Проблема дефектів оптичних поверхонь

Основні типи дефектів, що виявляються на поверхні лінз: **подряпини, тріщини, сколи, потертості, мікродефекти**. Навіть незначні пошкодження здатні погіршити оптичні характеристики виробу, викликати спотворення зображення та знизити загальну якість продукції до неприйнятного рівня.

Складність ручного контролю

Прозора поверхня лінзи створює відблиски та паразитні відбиття. Мікродефекти надзвичайно складно виявити візуально. Ручний контроль залежить від **досвіду оператора, рівня втоми та умов освітлення** — що призводить до низької стабільності результатів при великих обсягах виробництва.

Перехід до автоматизації

Комп'ютерний зір та цифрова обробка зображень дозволяють усунути людський фактор. Переваги автоматизованих систем:

- Стабільність та повторюваність аналізу
- Висока швидкість обробки (тисячі виробів/день)
- Безперервна робота в режимі 24/7
- Мінімізація людського фактора

Вимоги Industry 4.0

Сучасні виробничі системи вимагають: роботи в реальному часі, автоматичного виявлення та класифікації дефектів, сегментації об'єкта, формування звітності та повної інтеграції у виробничі процеси.

2

Проблеми предметної галузі та аналіз аналогів

Проблеми існуючих методів контролю

Людський фактор

Ручний контроль — основний метод на більшості підприємств. Висока суб'єктивність, втома оператора, помилки при тривалій роботі знижують стабільність результатів.

Складність виявлення мікродефектів

Подряпини, тріщини, плями важко виявити через відблиски та нерівномірне освітлення оптичних поверхонь. Точність залежить від досвіду інспектора.

Низька продуктивність

При масовому виробництві (тисячі виробів/день) ручний контроль стає вузьким місцем. Відсутній безперервний 24/7 режим та автоматичне сортування «придатні / браковані».

Відсутність аналітики

Немає системного збереження результатів, аналітики браку, звітності. Висока ціна помилок контролю тягне фінансові та технічні наслідки.

Аналіз існуючих програмних рішень

Промислові системи CV

- **Cognex** — висока точність, промисловий рівень, але дуже висока вартість
- **Keyence** — прості готові рішення, обмежена гнучкість налаштувань
- **Omnicon** — інтеграція з виробництвом, складна конфігурація
- **Basler** — орієнтація на камери, потребує власної логіки

AI / Deep Learning системи

- Висока точність у складних задачах
- Виявляють нетипові дефекти
- Потребують великих датасетів, обчислювальних ресурсів
- Складні у впровадженні для малих підприємств

Бібліотека CV (OpenCV)

- Потужний інструмент для обробки зображень
- Дає алгоритми фільтрації, контурів, геометрії
- Не є готовою системою контролю якості
- Потребує розробки інтерфейсу та бізнес-логіки

Висновок: Існуючі рішення орієнтовані або на великі підприємства, або потребують складної розробки. Відсутні доступні, прості та гнучкі системи для малих і середніх виробництв на основі комп'ютерного зору.

Постановка задачі

Актуальність задачі

Контроль якості оптичних ліній є критично важливим у сучасній промисловості. Навіть мікродефекти — подряпини, тріщини, сколи — суттєво впливають на якість оптики. Ручний контроль є ненадійним через людський фактор, а існуючі автоматизовані системи не відповідають вимогам малих і середніх виробництв за критеріями доступності та гнучкості.

Основна проблема

Існуючі методи контролю якості є повільними, залежними від оператора та недостатньо точними для виявлення мікродефектів. Складно автоматично виявляти подряпини, локальні неоднорідності поверхні в умовах відблисків і змінного освітлення. Відсутня доступна універсальна система для малих і середніх виробництв.

- ① Мета: розробити програмну систему автоматизованого аналізу зображень ліній із повним циклом обробки, класифікації та звітності.

Постановка задачі — необхідно розробити систему, яка:

Обробка зображень

Завантаження PNG, JPG, BMP; попередня обробка (фільтрація, grayscale, CLAHE); визначення меж лінії; аналіз геометрії об'єкта.

Виявлення та класифікація дефектів

Автоматичне виявлення подряпин, плям, артефактів; класифікація стану лінії (придатна / дефектна); підрахунок кількості дефектів; наочна візуалізація результатів.

Автоматизація моніторингу

Ручний та автоматичний режим роботи; моніторинг директорії incoming; автоматична обробка нових файлів; сортування у папки good / defects.

Звітність

Формування PDF-звітів; зображення, статистика та технічні дані; збереження та архівування історії результатів аналізу.

Архітектура програмного забезпечення

Система побудована за **модульним принципом**: кожен компонент виконує окрему функцію, обмін даними здійснюється через уніфіковані структури результатів. Архітектура забезпечує масштабованість, простоту підтримки та можливість розширення функціоналу.



GUI (PySide6) Взаємодія з оператором, відображення результатів, керування режимами роботи системи.	Image Processing Попередня обробка зображень, виявлення лінії та її меж, детекція дефектів, формування результату.
Folder Monitor Відстеження папки incoming, автоматичний запуск аналізу, фоновий режим (QThread, 24/7).	Report Generator Формування PDF-звітів: зображення, статистика дефектів, час і дата перевірки.
✔ Архітектурний принцип: Модульна архітектура забезпечує стабільну та масштабовану роботу системи автоматичного контролю якості. Обробка зображень, UI та файлова система є повністю незалежними компонентами.	

UML проєктування програмного забезпечення

UML використано для **формалізації поведінки та структури** системи автоматизованого контролю якості ліній, включаючи взаємодію користувача з GUI та внутрішніх модулів між собою.

Use Case Diagram Актор: Користувач (Оператор) - Завантажити зображення лінії - Запустити / зупинити автоматичний моніторинг - Переглянути результати аналізу в GUI - Згенерувати PDF-звіт Підсистеми: Модуль обробки зображень, Модуль моніторингу директорій, Модуль генерації PDF. Система підтримує ручний та автоматичний режими аналізу.	Sequence Diagram Сценарій 1 — Одиначна обробка: - Користувач обирає файл → GUI передає шлях - ImageProcessor: завантаження → аналіз → детекція - Результат повертається у GUI: зображення, кількість дефектів, статус Сценарій 2 — Автомоніторинг: FolderMonitor перевіряє incoming → новий файл → запускає аналіз → класифікація → переміщення (good/defect) → GUI оновлюється через сигнали (image_processed, status_changed). Обробка у QThread, GUI не блокується.	Class Diagram MainWindow — координує всю систему: завантаження, запуск моніторингу, оновлення UI, виклик звітів ImageProcessor — попередня обробка, виявлення лінії, аналіз дефектів, класифікація FolderMonitor — моніторинг incoming, QThread, автозапуск аналізу ReportGenerator — PDF з зображеннями, статистикою, метаданими ImageConverter / Utils — конвертація зображень, підготовка даних для UI Архітектурний принцип: розділення відповідальностей (Separation of Concerns), незалежність обробки, UI та файлової системи.
---	--	---

❏ UML-діаграми відображають повну архітектуру системи: сценарії взаємодії користувача, послідовність обробки даних та структуру основних програмних компонентів.

Алгоритми обробки зображень



1. Попередня обробка зображення

Завантаження (PNG, JPG, BMP) → перетворення у **grayscale** → зменшення шумів (**Gaussian Blur**) → підвищення контрасту за допомогою **CLAHE** (Contrast Limited Adaptive Histogram Equalization). Результат: покращена видимість мікродефектів, стабілізація освітлення.

2. Виявлення меж ліній

Власний алгоритм радіального аналізу від центру зображення: сканування по багатьох променях (0–360°), пошук максимального градієнту яскравості на кожному промені. Не використовується стандартний Hough Circle — алгоритм стійкий до нерівномірного освітлення та шумів.

3. Геометрична модель ліній

Фільтрація аномальних точок межі → побудова еліпсу, що апроксимує форму лінії. Враховує нахил камери, перспективні спотворення та неідеальну форму об'єкта.

4. Маска аналізу (ROI)

Створюється бінарна маска лінії та додаткова **inner mask** — безпечна зона аналізу. Краї лінії виключаються через високу концентрацію артефактів відбиття.

5. Детекція дефектів поверхні

Pipeline: нормалізація фону (Gaussian Blur + різниця з оригіналом) → виділення дрібних структур → **Canny Edge Detector** → морфологічні операції → **HoughLinesP** для виявлення лінійних дефектів (подряпин). Виконується фільтрація дублікатів ліній.

6. Підвищення точності

CLAHE підсилює локальний контраст; інверсія зображення (negative mode) покращує видимість слабких подряпин. Комбінований підхід підвищує загальну точність детекції дефектів.

7

Засоби розробки

Всі засоби розробки обрані з урахуванням вимог до автоматизованої системи комп'ютерного зору, обробки зображень у реальному часі та побудови desktop-застосунку промислового рівня.

Python — мова програмування

Основна мова розробки всієї системи. Використовується для реалізації алгоритмів комп'ютерного зору, обробки зображень, автоматизації та UI-логіки. Переваги: проста, велика екосистема бібліотек, висока швидкість розробки, підтримка наукових обчислень.

OpenCV + NumPy — комп'ютерний зір

OpenCV — основний інструмент для аналізу зображень: фільтрація, перетворення кольорів, детекція контурів, Canny, HoughLinesP. **NumPy** — робота з масивами пікселів, математичні операції. Разом реалізують весь pipeline обробки ліній від завантаження до виявлення дефектів.

PySide6 (Qt for Python) — графічний інтерфейс

Створення desktop GUI-застосунку: багатовкладковий інтерфейс для відображення етапів обробки, сигнали/слоти для взаємодії потоків (моніторинг + UI). Переваги: сучасний вигляд, стабільність, підтримка складних інтерактивних систем.

ReportLab — генерація PDF-звітів

Автоматичне формування документації результатів аналізу. Включає зображення етапів обробки, статистику дефектів, дату, час та статус перевірки. Забезпечує повну звітність без участі оператора.

OS / pathlib / Threading / Math

OS / pathlib — робота з файловою системою, моніторинг директорій (incoming, processed, defects). **QThread / Threading** — реалізація фонові обробки без блокування UI. **Math** — геометричні обчислення (радіуси, кути, параметри еліпсів).

8

Інтерфейс та фрагменти коду програми

Графічний інтерфейс системи

Інтерфейс реалізовано на **PySide6 (Qt Framework)**. Одновіконна архітектура забезпечує зручність оператора та мінімум дій для виконання аналізу.

Панель керування

Завантаження зображення, start/stop моніторингу, генерація PDF-звіту — в один клік.

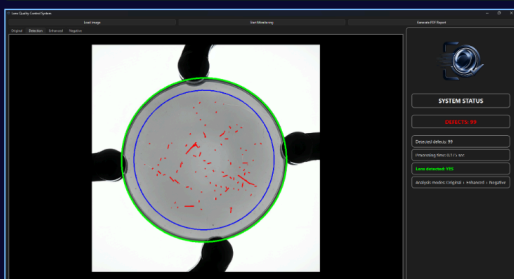
Вкладки результатів

Відображення: оригіналу, grayscale-покращення, негативного зображення, результату детекції дефектів.

Панель статусу

Індикація стану аналізу: кількість дефектів, час обробки, статус **OK / DEFECT**.

Система працює в реальному часі. Автоматизація без участі оператора — ключова перевага реалізованого рішення.



Фрагменти коду — реалізація алгоритмів

Unicode-safe завантаження зображення:

```
@staticmethod
def load_image_unicode(image_path):

    data = np.fromfile(image_path, dtype=np.uint8)

    return cv2.imdecode(data, cv2.IMREAD_COLOR)
```

Забезпечує коректне завантаження зображень із підтримкою кирилических шляхів та нестандартних символів у файловій системі.

Покращення контрасту (CLAHE):

```
@staticmethod
def enhance_contrast(gray):
    clahe = cv2.createCLAHE(
        clipLimit=3.0,
        tileGridSize=(8, 8)
    )
    return clahe.apply(gray)
```

Адаптивне підсилення контрасту для покращення видимості мікродефектів поверхні лінзи.

Радіальний аналіз — визначення меж лінзи:

```
for angle in np.linspace(0, 2*np.pi, RAY_COUNT):
    for r in range(20, max_radius):
        gradient = abs(value - prev)
        if gradient > EDGE_THRESHOLD:
            points.append(best_point)
```

Сканування по променях 0–360° для виявлення меж лінзи та аномалій поверхні за градієнтом яскравості.

9

Аналіз результатів

Загальний результат

- Створено програмну систему автоматизованого контролю якості лінз
- Реалізовано повний цикл: завантаження → обробка → аналіз → класифікація
- Система працює у **ручному та автоматичному** режимах
- Забезпечено безперервний моніторинг директорії (24/7)
- Автоматичне сортування у папки: **defects / processed**

Технічні результати алгоритму

- Стабільне виявлення меж лінзи (еліптична апроксимація)
- Детекція дефектів: подряпини, плями, локальні аномалії
- Застосовано: CLAHE, Canny, Gaussian Blur, HoughLinesP
- Фільтрація дублюючих дефектів, зменшення хибних спрацювань
- Стійкість до шуму та змінних умов освітлення

Функціональні результати

- Завантаження PNG, JPG, JPEG, BMP
- Автоматичний аналіз нових файлів у директорії
- Формування **PDF-звітів** з результатами аналізу
- Відображення оригіналу, оброблених зображень та результатів детекції
- Індикація статусу: **OK / DEFECT** у режимі реального часу

Практична цінність

- Підходить для промислового контролю якості малих і середніх виробництв
- Зменшує вплив людського фактора, підвищує швидкість перевірки
- Безперервна робота 24/7 без участі оператора
- Потенціал розширення: AI/ML моделі, відеопотоки, база даних результатів

10

Висновки

1 Досягнення мети роботи

Повністю реалізовано програмну систему автоматизованого контролю якості оптичних лінз. Система виконує аналіз зображень без участі оператора. Мету автоматизації первинного контролю якості досягнуто.

2 Реалізований функціонал

Завантаження та обробка PNG/JPG/BMP; виявлення лінзи та побудова геометричної моделі; детекція дефектів (подряпини, плями, аномалії); класифікація «придатний / дефектний»; автономіторинг 24/7; формування PDF-звітів.

3 Технології та методи

Python, OpenCV, NumPy — обробка зображень та комп'ютерний зір; CLAHE, Canny, HoughLinesP — детекція; еліптична апроксимація — геометрія; PySide6 — GUI; ReportLab — звітність.

1 Основні результати

Створено стабільну систему аналізу зображень у реальному часі. Зменшено вплив людського фактора. Забезпечено автоматичне сортування та наочну візуалізацію результатів для оператора.

2 Практична цінність

Система придатна для виробничих підприємств, лабораторій контролю якості та навчальних цілей. Може бути інтегрована у автоматизовані виробничі лінії Industry 4.0, знижує витрати на ручний контроль.

3 Перспективи розвитку

Впровадження машинного навчання для класифікації дефектів; інтеграція з промисловими камерами та відеопотоком; розширення бази даних результатів; оптимізація для масштабованих виробничих систем.

✓ **Підсумок:** Програмна система повністю відповідає поставленим вимогам і може бути використана як основа для промислового рішення автоматизованого контролю якості оптичних виробів.

ДОДАТОК В

Специфікація програмного забезпечення

Програмне забезпечення системи автоматизованого контролю якості оптичних лінз призначене для автоматичного виявлення поверхневих дефектів оптичних елементів на основі методів цифрової обробки зображень та комп'ютерного зору. Система забезпечує аналіз цифрових зображень лінз, локалізацію області контролю, пошук дефектів поверхні, класифікацію результатів контролю, автоматичне сортування зображень та формування звітної документації. Програмне забезпечення реалізоване мовою Python та функціонує як багатомодульний настільний застосунок із графічним інтерфейсом користувача. Для реалізації алгоритмів комп'ютерного зору використовується бібліотека OpenCV, для математичної обробки даних — NumPy, для побудови графічного інтерфейсу — PySide6, а для генерації звітної документації — бібліотека ReportLab.

Архітектура системи побудована за модульним принципом і включає:

- модуль обробки зображень;
- модуль графічного інтерфейсу користувача;
- модуль автоматичного моніторингу директорій;
- модуль формування звітів;
- модуль конвертації зображень для GUI-відображення.

Основною функцією програмного забезпечення є автоматичне виявлення дефектів поверхні оптичної лінзи. Аналіз виконується шляхом послідовної обробки зображення із використанням алгоритмів покращення контрасту, локалізації меж лінзи, побудови області аналізу, фільтрації шумів, виділення контурів та пошуку лінійних дефектів. Система підтримує завантаження графічних файлів форматів PNG, JPG, JPEG та BMP. Завантаження зображень реалізовано із підтримкою Unicode-шляхів, що забезпечує коректну роботу з

файлами та каталогами, назви яких містять символи національних алфавітів. Для цього використовується механізм зчитування байтових даних із подальшим декодуванням засобами OpenCV. Після завантаження виконується попередня обробка зображення. Кольорове зображення перетворюється у напівтонове представлення, після чого застосовується алгоритм адаптивного вирівнювання гістограми CLAHE. Метод CLAHE використовується для локального покращення контрасту та компенсації нерівномірного освітлення поверхні. Параметри алгоритму включають обмеження контрасту та поділ зображення на локальні області обробки. Застосування даного методу дозволяє підвищити видимість мікродфектів та стабілізувати результати подальшого аналізу. Для визначення області розташування лінзи використовується алгоритм радіального сканування. Система формує набір променів, які виходять із центральної точки зображення та проходять у всіх напрямках по колу. Уздовж кожного променя виконується аналіз зміни інтенсивності пікселів. Точка максимального перепаду яскравості інтерпретується як потенційна межа лінзи. Для усунення шумових переходів використовується порогове значення градієнта яскравості. Усі знайдені граничні точки проходять процедуру статистичної фільтрації. Для кожної точки обчислюється відстань до центру мас множини точок, після чого визначається медіанне значення радіуса. Точки, відхилення яких перевищує допустимий поріг, виключаються із подальшого аналізу. Такий підхід дозволяє усунути аномальні значення, спричинені шумами, відблисками або локальними неоднорідностями освітлення. Після завершення фільтрації виконується апроксимація форми лінзи еліпсом. Для цього використовується метод найменших квадратів, реалізований у функції апроксимації еліптичного контуру OpenCV. У результаті визначаються координати центра лінзи, довжини головних осей та кут нахилу еліпса. Якщо кількість валідних точок є недостатньою або апроксимація не може бути виконана, система формує повідомлення про помилку локалізації лінзи та завершує процедуру аналізу. Після успішної локалізації формується система вкладених масок області аналізу. Повна маска лінзи використовується для виділення всієї робочої області оптичного елемента. Внутрішня безпечна маска формується

шляхом пропорційного зменшення осей еліпса та використовується для виключення периферійної області, де можуть виникати крайові спотворення, тіні або засвітки. Окремо формується центральна область пошуку дефектів, яка обмежує аналіз внутрішньою частиною лінзи та підвищує стабільність детекції. Для компенсації неоднорідностей освітлення використовується алгоритм нормалізації фону. Метод базується на побудові сильно розмитої копії зображення за допомогою гаусового фільтра великого розміру. Після цього виконується обчислення абсолютної різниці між вихідним та розмитим зображенням. Отриманий результат нормалізується у повний діапазон яскравості. Використання такого підходу дозволяє підсилити локальні поверхневі аномалії та зменшити вплив глобальних змін освітлення.

Виявлення дефектів реалізується у двох незалежних режимах аналізу:

- аналіз стандартного напівтонового зображення;
- аналіз негативного представлення зображення.

Для негативного режиму виконується інверсія яскравості пікселів, що дозволяє ефективніше виявляти світлі дефекти на темному фоні або слабоконтрастні мікропошкодження поверхні. У кожному режимі виконується згладжування зображення методом гаусового розмиття для зменшення шумів. Після цього застосовується детектор границь Canny, який забезпечує виділення контурів потенційних дефектів. Для підвищення цілісності контурів використовується морфологічна операція замикання, що усуває дрібні розриви та об'єднує фрагментовані сегменти дефектів. Пошук подряпин та мікротріщин виконується за допомогою ймовірнісного перетворення Хафа. Алгоритм дозволяє виділяти лінійні структури на бінарному зображенні контурів. Для кожного знайденого сегмента обчислюється його геометрична довжина. Додатково виконується перевірка просторової близькості між сегментами з метою усунення дублювання результатів. Якщо відстань між початковими точками двох ліній є меншою за встановлений поріг, лінії вважаються повторними та не враховуються повторно. Усі підтверджені дефекти відображаються на результуючому

зображенні у вигляді червоних ліній. Межа лінзи позначається зеленим еліпсом, а безпечна зона аналізу — синім еліпсом. Такий підхід забезпечує візуальний контроль правильності локалізації та результатів аналізу. Результатом роботи модуля обробки є структура даних, що містить:

- оригінальне зображення;
- покращене напівтонове представлення;
- негативне представлення;
- результуюче зображення з позначеними дефектами;
- кількість виявлених дефектів;
- текстовий статус результату контролю.

Графічний інтерфейс користувача реалізований засобами бібліотеки PySide6 та забезпечує взаємодію оператора із системою контролю. Інтерфейс має багатовкладкову структуру та включає вкладки:

- original;
- detection;
- enhanced;
- negative.

Кожна вкладка відображає відповідний режим обробки зображення. Масштабування зображень виконується із збереженням пропорцій та використанням згладжування для покращення візуальної якості. Головне вікно системи містить панель керування, інформаційну область та область візуалізації результатів. Панель керування забезпечує:

- завантаження зображень;
- запуск автоматичного моніторингу;

- зупинку автоматичного моніторингу;
- генерацію PDF-звітів.

Інформаційна панель системи відображає:

- поточний статус системи;
- кількість виявлених дефектів;
- час обробки зображення;
- статус виявлення лінзи;
- активні режими аналізу.

Колірна індикація результатів використовується для швидкого візуального сприйняття стану контролю. У разі відсутності дефектів статус відображається зеленим кольором. У разі виявлення дефектів або помилки локалізації використовується червоне виділення. Система підтримує автоматичний режим контролю на основі файлового моніторингу. Для цього реалізований окремий потік виконання FolderMonitor, який безперервно відстежує появу нових файлів у директорії images/incoming. Після виявлення нового зображення виконується його автоматичний аналіз без участі оператора. За результатами аналізу система автоматично виконує сортування файлів:

- файли без дефектів переміщуються до директорії images/processed;
- файли з дефектами переміщуються до директорії images/defects.

Моніторинг реалізований у багатопоточному режимі, що забезпечує безперервну роботу інтерфейсу користувача під час виконання автоматичного контролю. Для обміну даними між потоками використовуються сигнали та слоти Qt. Формування звітної документації виконується у форматі PDF. Генерація звіту здійснюється автоматично після завершення аналізу за запитом користувача. Документ створюється у форматі A4 та містить:

- назву файла;
- дату та час аналізу;
- кількість виявлених дефектів;
- системний статус;
- інформацію про успішність локалізації лінзи;
- результати всіх режимів обробки зображення.

У звіті використовуються таблиці, текстові блоки та графічні матеріали. Для формування документа застосовується бібліотека ReportLab. Зображення тимчасово експортуються у графічні файли та автоматично видаляються після завершення побудови документа. До основних функціональних характеристик програмного забезпечення належать:

- автоматичне завантаження та обробка зображень;
- покращення контрасту;
- локалізація області лінзи;
- апроксимація форми лінзи;
- формування області аналізу;
- нормалізація фону;
- багаторежимне виявлення дефектів;
- автоматичний підрахунок дефектів;
- візуалізація результатів контролю;
- автоматичне сортування файлів;
- генерація PDF-звітів;

- підтримка багатопоточності.

До нефункціональних характеристик програмного забезпечення належать:

- модульна архітектура;
- масштабованість структури системи;
- підтримка Unicode-шляхів;
- підтримка режиму реального часу;
- стабільність тривалої безперервної роботи;
- сумісність із операційними системами Windows;
- можливість інтеграції із системами машинного зору;
- можливість адаптації до промислових камер та автоматизованих виробничих ліній;
- розширюваність алгоритмів аналізу;
- підтримка автоматизованого контролю якості у виробничому середовищі.